B.TECH CSE | DATABASE MANAGEMENT SYSTEM

Database Management System Study Notes

Clear concepts, ER diagrams, SQL, and exam practice

A module-wise guide to the syllabus supplied for these notes

Simple explanation first. Formal definition next. Then a worked example.

Use the diagrams, SQL traces, and dependency checks as active revision.

STUDENT EDITION | 139 PAGES | 2 MODULES

A clear route through DBMS

How to study each topic

- Start with the plain-language definition; then read the formal rule and notation.
- For ER topics, draw the entities/relationships and mark keys, cardinality, and participation.
- For algebra/calculus, write the intermediate result; for SQL, predict affected rows before checking the example.
- For normalization, list candidate keys and dependencies before applying a normal-form test.
- Use the exam/viva boxes and module revision section to practice explaining concepts aloud.

Database used in examples

- Student(SID, Name, Dept, GPA); Course(CID, Title); Enrollment(SID, CID, Grade).
- SID and CID identify rows in their parent tables; Enrollment commonly uses the composite key (SID, CID).
- SQL examples use small, illustrative schemas. Adapt data types and syntax to the database product used in class.

QUICK RECALL

Read the example as a trace: what are the inputs, what rule runs, and what exact result follows?

MODULE-I - what to study and in what order

Module goal

Build the database vocabulary, understand ER design, then reason about relational tables and formal query/constraint concepts.

Syllabus map - exact topic order

- Introduction: An Overview of Database Management System; Database System vs File System; Database System Concept and Architecture; Data Model, Schema, and Instances; Data Independence; Database Language and Interfaces; Data Definition Language (DDL); Data Manipulation Language (DML); Overall Database Structure.
- Data Modeling: ER Data Model; Notation for ER Diagram; Mapping Constraints; Super Key, Candidate Key, and Primary Key; Generalization; Aggregation; Reduction of an ER Diagram to Tables; Extended ER (EER) Model; Relationship of Higher Degree.
- Relational Data Model: Relational Data Model Concepts; Integrity Constraints; Entity Integrity; Referential Integrity; Key Constraints; Domain Constraints; Relational Algebra; Relational Calculus; Tuple Relational Calculus; Domain Relational Calculus.

Recommended study order

- Read database purpose and architecture before studying schemas and languages.
- Draw ER/EER structures and map them to relations.
- Finish with keys, integrity rules, algebra, and calculus.

QUICK RECALL

Before moving on, map one M:N ER relationship to a table and explain a foreign key.

TOPIC 01 / 60 | Introduction CONCEPT AND DEFINITIONS

An Overview of Database Management System

Simple definition

A database stores related information in an organized way. A DBMS is the software that lets people define, store, search, update, and protect that information.

Formal / technical definition

A database management system is a software system that defines database schemas, stores persistent data, processes queries and updates, controls concurrent access, and provides recovery and authorization mechanisms.

Notation and how to read it

database = organized related data; DBMS = software that manages it; database system = database + DBMS + users/apps + hardware

How it works

- Identify the database user or application and its data need.
- Place the idea at the correct level: stored data, schema, DBMS component, language, or interface.
- State what changes, what is protected, and what result the user receives.

Quick map / trace / syntax

```
User / application -> SQL or interface -> DBMS -> stored database
| security, concurrency, recovery
```

QUICK RECALL

Define both database and DBMS, then state the services the software provides.

TOPIC 01 / 60 | Introduction

CODE EXAMPLE AND EXAM HELP

An Overview of Database Management System

Worked code example and output

```
Example: a college DBMS stores Student, Course, and Enrollment records. A query can find
students in a course without manually opening many separate files.
```

Useful distinction / limitation

A DBMS is not the database itself: the database is the stored collection; the DBMS is the managing software.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Define both database and DBMS, then state the services the software provides.
- Likely follow-up: A DBMS is not the database itself: the database is the stored collection; the DBMS is the managing software.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Define both database and DBMS, then state the services the software provides.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 02 / 60 | Introduction

CONCEPT AND DEFINITIONS

Database System vs File System

Simple definition

With separate files, every program manages its own data. A DBMS gives multiple users and programs a shared, controlled way to work with related data.

Formal / technical definition

A file-processing system stores application data in independent files whose formats and operations are embedded in programs. A database system centralizes schema and data management and supports controlled sharing, integrity, concurrency, and recovery.

Notation and how to read it

DBMS advantages: reduced redundancy, controlled inconsistency, sharing, integrity, security, backup/recovery, data independence. Costs: setup, administration, and resource overhead.

How it works

- Identify the database user or application and its data need.
- Place the idea at the correct level: stored data, schema, DBMS component, language, or interface.
- State what changes, what is protected, and what result the user receives.

Quick map / trace / syntax

```
File-per-program -> duplicate copies -> update anomalies
Shared DBMS -> common schema -> governed reads/writes
```

QUICK RECALL

Compare data redundancy, sharing, independence, integrity, security, and recovery.

TOPIC 02 / 60 | Introduction

CODE EXAMPLE AND EXAM HELP

Database System vs File System

Worked code example and output

```
File approach: admissions.txt and fees.txt may repeat a student's address. DBMS approach:
Student(SID, Address) stores it once; related tables use SID.
```

Useful distinction / limitation

A DBMS does not eliminate all duplication or guarantee perfect security automatically; design and configuration still matter.

Common mistakes to avoid

- Do not claim a DBMS removes every duplicate or guarantees security automatically.
- Compare the same criteria: redundancy, sharing, integrity, access, and recovery.

Quick viva and follow-up cues

- Core answer: Compare data redundancy, sharing, independence, integrity, security, and recovery.
- Likely follow-up: A DBMS does not eliminate all duplication or guarantee perfect security automatically; design and configuration still matter.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Compare data redundancy, sharing, independence, integrity, security, and recovery.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 03 / 60 | Introduction

CONCEPT AND DEFINITIONS

Database System Concept and Architecture

Simple definition

Database architecture explains how users see data and how the system stores and processes it behind the scenes.

Formal / technical definition

The common three-schema architecture separates external views, a conceptual schema, and an internal schema. Mappings connect these levels. A DBMS implementation may also be described by its query processor, storage manager, transaction/recovery services, and catalog.

Notation and how to read it

External schema -> user-specific view

Conceptual schema -> logical database structure

Internal schema -> physical storage details

How it works

- Identify the database user or application and its data need.
- Place the idea at the correct level: stored data, schema, DBMS component, language, or interface.
- State what changes, what is protected, and what result the user receives.

Quick map / trace / syntax

```
User views (external)
| external-conceptual mapping
Conceptual schema
| conceptual-internal mapping
Internal storage
```

QUICK RECALL

Draw and label all three levels, then explain what each level hides.

TOPIC 03 / 60 | Introduction

CODE EXAMPLE AND EXAM HELP

Database System Concept and Architecture

Worked code example and output

```
A payroll user sees employee names and salaries; the conceptual model has Employee and
Department; the internal level decides pages, indexes, and files.
```

Useful distinction / limitation

Three-schema architecture is a way to reason about separation and independence; it does not require exactly three physical software programs.

Common mistakes to avoid

- Do not swap conceptual and internal levels.
- Explain external, conceptual, and internal schemas with the mappings between them.

Quick viva and follow-up cues

- Core answer: Draw and label all three levels, then explain what each level hides.
- Likely follow-up: Three-schema architecture is a way to reason about separation and independence; it does not require exactly three physical software programs.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Draw and label all three levels, then explain what each level hides.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 04 / 60 | Introduction

CONCEPT AND DEFINITIONS

Data Model, Schema, and Instances

Simple definition

A data model is a set of ideas for describing data. A schema is the planned structure. An instance is the actual data at a particular time.

Formal / technical definition

A data model defines constructs, relationships, constraints, and operations. A database schema is its specification; a database state/instance is the set of stored values conforming to that schema at a moment.

Notation and how to read it

Model = description vocabulary

Schema = relatively stable design

Instance/state = current rows/values

How it works

- Identify the database user or application and its data need.
- Place the idea at the correct level: stored data, schema, DBMS component, language, or interface.
- State what changes, what is protected, and what result the user receives.

Quick map / trace / syntax

```
Schema: Student(SID, Name, Dept)
Instance at 10:00 -> 2 rows
Instance after INSERT -> 3 rows
```

QUICK RECALL

Give one schema and two different valid instances to show the distinction.

TOPIC 04 / 60 | Introduction

CODE EXAMPLE AND EXAM HELP

Data Model, Schema, and Instances

Worked code example and output

```
Schema: Student(SID, Name, Dept). Instance: (17, 'Asha', 'CSE'), (18, 'Ravi', 'ECE').
Adding a row changes the instance, not necessarily the schema.
```

Useful distinction / limitation

Schema can also change through DDL; it is relatively stable, not permanently fixed.

Common mistakes to avoid

- Schema is structure; instance is current data.
- A row insertion usually changes the instance, not the schema.

Quick viva and follow-up cues

- Core answer: Give one schema and two different valid instances to show the distinction.
- Likely follow-up: Schema can also change through DDL; it is relatively stable, not permanently fixed.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Give one schema and two different valid instances to show the distinction.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 05 / 60 | Introduction

CONCEPT AND DEFINITIONS

Data Independence

Simple definition

Data independence means changing one level of database design without forcing every higher-level view or program to change.

Formal / technical definition

Physical data independence permits changes to the internal schema without changing the conceptual or external schemas. Logical data independence permits changes to the conceptual schema without requiring changes to external schemas; it is generally harder to achieve.

Notation and how to read it

Physical: internal change -> conceptual view unchanged

Logical: conceptual change -> external view/application preserved where mappings allow

How it works

- Identify the database user or application and its data need.
- Place the idea at the correct level: stored data, schema, DBMS component, language, or interface.
- State what changes, what is protected, and what result the user receives.

Quick map / trace / syntax

```
External schemas
| logical independence
Conceptual schema
| physical independence
Internal schema
```

QUICK RECALL

State which schema changes and which level is meant to remain unaffected.

TOPIC 05 / 60 | Introduction

CODE EXAMPLE AND EXAM HELP

Data Independence

Worked code example and output

```
Adding an index to speed up Student lookup is usually a physical change; a view can hide a newly added column from an existing application.
```

Useful distinction / limitation

Physical independence concerns storage; logical independence concerns the logical structure and user views.

Common mistakes to avoid

- Physical independence concerns internal storage; logical independence concerns conceptual structure.
- Do not promise every application is unaffected by every schema change.

Quick viva and follow-up cues

- Core answer: State which schema changes and which level is meant to remain unaffected.
- Likely follow-up: Physical independence concerns storage; logical independence concerns the logical structure and user views.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: State which schema changes and which level is meant to remain unaffected.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 06 / 60 | Introduction

CONCEPT AND DEFINITIONS

Database Language and Interfaces

Simple definition

Users work with a DBMS through languages and interfaces: SQL, forms, graphical tools, APIs, or administrative consoles.

Formal / technical definition

A database language provides constructs to define schemas, query/update data, control access, and manage transactions. Interfaces expose these operations through command lines, GUIs, embedded APIs, or higher-level applications.

Notation and how to read it

Human/application -> interface or API -> database language -> DBMS

How it works

- Identify the database user or application and its data need.
- Place the idea at the correct level: stored data, schema, DBMS component, language, or interface.
- State what changes, what is protected, and what result the user receives.

Quick map / trace / syntax

```
Form / SQL client / program API
-> DBMS request
<- result / status
```

QUICK RECALL

Separate the language from the user interface and mention one user group for each interface.

TOPIC 06 / 60 | Introduction

CODE EXAMPLE AND EXAM HELP

Database Language and Interfaces

Worked code example and output

```
A student portal may use a form; its application sends a parameterized SQL query through a database driver. An administrator may use a SQL console.
```

Useful distinction / limitation

SQL is the language; a particular SQL client or web form is an interface that sends requests.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Separate the language from the user interface and mention one user group for each interface.
- Likely follow-up: SQL is the language; a particular SQL client or web form is an interface that sends requests.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Separate the language from the user interface and mention one user group for each interface.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 07 / 60 | Introduction

CONCEPT AND DEFINITIONS

Data Definition Language (DDL)

Simple definition

DDL statements create or change database structures such as tables, constraints, and views.

Formal / technical definition

Data Definition Language is the schema-definition portion of a database language. Typical statements include CREATE, ALTER, and DROP; exact transaction behavior can depend on the DBMS.

Notation and how to read it

CREATE -> define object

ALTER -> change definition

DROP -> remove object definition (and usually its data)

How it works

- Identify the database user or application and its data need.
- Place the idea at the correct level: stored data, schema, DBMS component, language, or interface.
- State what changes, what is protected, and what result the user receives.

Quick map / trace / syntax

```
DDL statement -> catalog/schema metadata changes -> DBMS validates constraints and stores definition
```

QUICK RECALL

Identify whether a command changes structure or changes row values.

TOPIC 07 / 60 | Introduction

CODE EXAMPLE AND EXAM HELP

Data Definition Language (DDL)

Worked code example and output

```
SQL:
CREATE TABLE Student (
SID INTEGER PRIMARY KEY,
Name VARCHAR(60) NOT NULL
);
ALTER TABLE Student ADD Dept VARCHAR(20);
```

Useful distinction / limitation

DROP TABLE is destructive. Some systems implicitly commit certain DDL statements; do not assume transaction behavior is identical across vendors.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Identify whether a command changes structure or changes row values.
- Likely follow-up: DROP TABLE is destructive. Some systems implicitly commit certain DDL statements; do not assume transaction behavior is identical across vendors.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Identify whether a command changes structure or changes row values.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 08 / 60 | Introduction

CONCEPT AND DEFINITIONS

Data Manipulation Language (DML)

Simple definition

DML works with the rows inside tables: it retrieves, adds, changes, or removes data.

Formal / technical definition

Data Manipulation Language includes statements for querying and modifying database contents. Common operations are SELECT, INSERT, UPDATE, and DELETE; classification of SELECT as DML or DQL varies by textbook.

Notation and how to read it

INSERT -> add rows; UPDATE -> change selected rows; DELETE -> remove selected rows; SELECT -> retrieve rows

How it works

- Identify the database user or application and its data need.
- Place the idea at the correct level: stored data, schema, DBMS component, language, or interface.
- State what changes, what is protected, and what result the user receives.

Quick map / trace / syntax

```
DML statement + conditions -> DBMS checks permissions/constraints -> affected rows or result set
```

QUICK RECALL

Explain the target table, affected rows, and role of a WHERE condition.

TOPIC 08 / 60 | Introduction

CODE EXAMPLE AND EXAM HELP

Data Manipulation Language (DML)

Worked code example and output

```
SQL:
INSERT INTO Student (SID, Name) VALUES (17, 'Asha');
UPDATE Student SET Name='Asha Y' WHERE SID=17;
SELECT SID, Name FROM Student;
```

Useful distinction / limitation

An UPDATE or DELETE without a WHERE clause can affect every row in the table.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Explain the target table, affected rows, and role of a WHERE condition.
- Likely follow-up: An UPDATE or DELETE without a WHERE clause can affect every row in the table.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Explain the target table, affected rows, and role of a WHERE condition.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 09 / 60 | Introduction

CONCEPT AND DEFINITIONS

Overall Database Structure

Simple definition

A DBMS has parts that understand requests, find data, manage storage, enforce rules, and recover after failures.

Formal / technical definition

A typical database system includes a query processor (parser, optimizer, executor), storage manager (file, buffer, index managers), transaction/concurrency and recovery components, authorization/integrity services, and a system catalog.

Notation and how to read it

Query processor | storage manager | transaction manager | recovery manager | authorization/integrity | catalog

How it works

- Identify the database user or application and its data need.
- Place the idea at the correct level: stored data, schema, DBMS component, language, or interface.
- State what changes, what is protected, and what result the user receives.

Quick map / trace / syntax

```
SQL request -> parse/validate -> optimize -> execute -> buffer/files/indexes
^ catalog / rules / privileges
```

QUICK RECALL

Trace one query from parsing through execution and identify where metadata is consulted.

TOPIC 09 / 60 | Introduction

CODE EXAMPLE AND EXAM HELP

Overall Database Structure

Worked code example and output

```
For SELECT Dept FROM Student WHERE SID=17, the parser checks syntax, the optimizer selects an access plan, and the executor obtains rows through storage services.
```

Useful distinction / limitation

Component names differ across implementations; focus on each component's responsibility.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Trace one query from parsing through execution and identify where metadata is consulted.
- Likely follow-up: Component names differ across implementations; focus on each component's responsibility.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Trace one query from parsing through execution and identify where metadata is consulted.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 10 / 60 | Data Modeling

CONCEPT AND DEFINITIONS

ER Data Model

Simple definition

The Entity-Relationship model is a picture-based way to plan a database: it shows things we store, their properties, and how they connect.

Formal / technical definition

The ER model represents a conceptual schema using entity types, attributes, relationship types, and structural constraints. An entity is a distinguishable real-world object; an entity set/type describes similar entities.

Notation and how to read it

Entity type + attributes; relationship type + participating entity types; constraints describe allowed participation/cardinality.

How it works

- Read the requirement and identify entity types, attributes, and relationship meaning.
- Mark keys and minimum/maximum participation before drawing the final diagram.
- Check whether the diagram captures optionality, relationship attributes, and degree.

Quick map / trace / syntax

```
[STUDENT] --< ENROLLS >-- [COURSE]
SID, Name CID, Title
```

QUICK RECALL

Name entity types, key attributes, relationship, and any participation/cardinality assumptions.

TOPIC 10 / 60 | Data Modeling

CODE EXAMPLE AND EXAM HELP

ER Data Model

Worked code example and output

```
College example: Student(SID, Name) -- ENROLLS -- Course(CID, Title). A particular student and course are entity instances; ENROLLS is a relationship instance.
```

Useful distinction / limitation

A relationship is not always a separate entity. Use an associative entity when the relationship has attributes or needs its own identity.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Name entity types, key attributes, relationship, and any participation/cardinality assumptions.
- Likely follow-up: A relationship is not always a separate entity. Use an associative entity when the relationship has attributes or needs its own identity.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Name entity types, key attributes, relationship, and any participation/cardinality assumptions.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 11 / 60 | Data Modeling

CONCEPT AND DEFINITIONS

Notation for ER Diagram

Simple definition

ER notation uses shapes and labels so a diagram tells us which parts are entities, attributes, and relationships.

Formal / technical definition

In common Chen notation, entity types are rectangles, relationship types are diamonds, attributes are ovals, key attributes are underlined, and weak entities use double rectangles. Crow's-foot notation uses line-end symbols to show cardinality; notation varies by convention.

Notation and how to read it

Chen: rectangle=entity, diamond=relationship, oval=attribute, double rectangle=weak entity, double oval=multivalued attribute

How it works

- Read the requirement and identify entity types, attributes, and relationship meaning.
- Mark keys and minimum/maximum participation before drawing the final diagram.
- Check whether the diagram captures optionality, relationship attributes, and degree.

Quick map / trace / syntax

```
[STUDENT] -- <ENROLLS> -- [COURSE]
(SID) (CID)
key attributes are underlined in a drawn ERD
```

QUICK RECALL

Label entities, relationships, keys, and line constraints clearly; include a legend if notation could be ambiguous.

TOPIC 11 / 60 | Data Modeling

CODE EXAMPLE AND EXAM HELP

Notation for ER Diagram

Worked code example and output

```
STUDENT rectangle with underlined SID and Name oval; diamond ENROLLS connects STUDENT and
COURSE; label 1:N or M:N beside lines.
```

Useful distinction / limitation

Do not mix Chen and crow's-foot symbols without telling the reader which notation is being used.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Label entities, relationships, keys, and line constraints clearly; include a legend if notation could be ambiguous.
- Likely follow-up: Do not mix Chen and crow's-foot symbols without telling the reader which notation is being used.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Label entities, relationships, keys, and line constraints clearly; include a legend if notation could be ambiguous.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 12 / 60 | Data Modeling

CONCEPT AND DEFINITIONS

Mapping Constraints

Simple definition

Mapping constraints say how many entities can be connected and whether taking part is optional or required.

Formal / technical definition

Cardinality ratios constrain relationship participation as 1:1, 1:N, or M:N. Participation constraints are total/mandatory or partial/optional. Min-max notation gives a lower and upper participation bound for each entity type.

Notation and how to read it

1:1, 1:N, M:N; min-max examples: Employee (1,1), Department (0,N)

How it works

- Read the requirement and identify entity types, attributes, and relationship meaning.
- Mark keys and minimum/maximum participation before drawing the final diagram.
- Check whether the diagram captures optionality, relationship attributes, and degree.

Quick map / trace / syntax

```
DEPARTMENT (0,N) -- employs -- EMPLOYEE (1,1)
A department may have zero employees; each employee has exactly one department.
```

QUICK RECALL

State both maximum relationship count and optional/mandatory participation for each side.

TOPIC 12 / 60 | Data Modeling

CODE EXAMPLE AND EXAM HELP

Mapping Constraints

Worked code example and output

```
A Department employs many Employees, but each Employee belongs to one Department:
Department 1:N Employee. If every employee must belong to one, Employee participation is
total.
```

Useful distinction / limitation

Cardinality and participation answer different questions: maximum count vs whether participation may be absent.

Common mistakes to avoid

- Cardinality is a maximum-count rule; participation is optional vs mandatory.
- State the constraint on each side of the relationship.

Quick viva and follow-up cues

- Core answer: State both maximum relationship count and optional/mandatory participation for each side.
- Likely follow-up: Cardinality and participation answer different questions: maximum count vs whether participation may be absent.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: State both maximum relationship count and optional/mandatory participation for each side.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 13 / 60 | Data Modeling

CONCEPT AND DEFINITIONS

Super Key, Candidate Key, and Primary Key

Simple definition

A key identifies a row. A super key may contain extra columns; a candidate key is minimal; one candidate key is selected as the primary key.

Formal / technical definition

A superkey is an attribute set whose values uniquely identify each tuple. A candidate key is a minimal superkey. A primary key is the chosen candidate key; alternate candidate keys remain unique keys but are not selected as primary.

Notation and how to read it

K superkey if K -> all attributes; candidate key = minimal superkey; primary key = selected candidate key

How it works

- Read the requirement and identify entity types, attributes, and relationship meaning.
- Mark keys and minimum/maximum participation before drawing the final diagram.
- Check whether the diagram captures optionality, relationship attributes, and degree.

Quick map / trace / syntax

```
{SID} -> unique and minimal
{Email} -> unique and minimal
{SID, Name} -> unique but not minimal
```

QUICK RECALL

For a candidate-key question, prove uniqueness and test minimality by removing each attribute.

TOPIC 13 / 60 | Data Modeling

CODE EXAMPLE AND EXAM HELP

Super Key, Candidate Key, and Primary Key

Worked code example and output

```
Student(SID, Email, Name): if SID and Email are each unique, {SID}, {Email} are candidate keys; {SID, Name} is a superkey but not minimal. Choose SID as primary key.
```

Useful distinction / limitation

Minimal means no proper subset still uniquely identifies rows; it does not mean the key has the fewest characters.

Common mistakes to avoid

- A superkey can contain unnecessary attributes; a candidate key is minimal.
- A primary key is one selected candidate key; alternate candidate keys remain unique.

Quick viva and follow-up cues

- Core answer: For a candidate-key question, prove uniqueness and test minimality by removing each attribute.
- Likely follow-up: Minimal means no proper subset still uniquely identifies rows; it does not mean the key has the fewest characters.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: For a candidate-key question, prove uniqueness and test minimality by removing each attribute.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 14 / 60 | Data Modeling

CONCEPT AND DEFINITIONS

Generalization

Simple definition

Generalization combines several similar lower-level entity types into one common parent type.

Formal / technical definition

Generalization is a bottom-up abstraction in the EER model. Common attributes and relationships are placed in a superclass; specialized entity types become subclasses that inherit applicable properties.

Notation and how to read it

CAR is-a VEHICLE; TRUCK is-a VEHICLE; subclasses inherit superclass properties

How it works

- Read the requirement and identify entity types, attributes, and relationship meaning.
- Mark keys and minimum/maximum participation before drawing the final diagram.
- Check whether the diagram captures optionality, relationship attributes, and degree.

Quick map / trace / syntax

```
VEHICLE(RegNo, Make)
/ \
CAR TRUCK
SeatCount LoadLimit
```

QUICK RECALL

Identify common superclass attributes and subclass-specific attributes.

TOPIC 14 / 60 | Data Modeling

CODE EXAMPLE AND EXAM HELP

Generalization

Worked code example and output

```
CAR and TRUCK can be generalized to VEHICLE with shared RegNo and Make; CAR keeps
SeatCount and TRUCK keeps LoadLimit.
```

Useful distinction / limitation

A subclass membership is an is-a relationship. It should reflect shared meaning, not just two tables that happen to have similar columns.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Identify common superclass attributes and subclass-specific attributes.
- Likely follow-up: A subclass membership is an is-a relationship. It should reflect shared meaning, not just two tables that happen to have similar columns.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Identify common superclass attributes and subclass-specific attributes.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 15 / 60 | Data Modeling

CONCEPT AND DEFINITIONS

Aggregation

Simple definition

Aggregation lets an ER model treat a relationship and its participating entities as one higher-level object so another relationship can refer to it.

Formal / technical definition

Aggregation is an abstraction that encapsulates a relationship set together with its participating entity sets, allowing the aggregate to participate in another relationship.

Notation and how to read it

Aggregation: (Employee -- WORKS_ON -- Project) -- MONITORED_BY -- Manager

How it works

- Read the requirement and identify entity types, attributes, and relationship meaning.
- Mark keys and minimum/maximum participation before drawing the final diagram.
- Check whether the diagram captures optionality, relationship attributes, and degree.

Quick map / trace / syntax

```
[EMPLOYEE] --< WORKS_ON >-- [PROJECT]
(WORKS_ON assignment)
|
MONITORED_BY [MANAGER]
```

QUICK RECALL

Explain what is being aggregated and why the outer relationship cannot be modeled as a simpler direct link.

TOPIC 15 / 60 | Data Modeling

CODE EXAMPLE AND EXAM HELP

Aggregation

Worked code example and output

```
An Employee WORKS_ON a Project. A Manager MONITORS that particular assignment; MONITORS
relates Manager to the aggregated WORKS_ON relationship, not merely to Project.
```

Useful distinction / limitation

Use aggregation when another fact concerns the relationship instance itself; otherwise an ordinary relationship may be enough.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Explain what is being aggregated and why the outer relationship cannot be modeled as a simpler direct link.
- Likely follow-up: Use aggregation when another fact concerns the relationship instance itself; otherwise an ordinary relationship may be enough.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Explain what is being aggregated and why the outer relationship cannot be modeled as a simpler direct link.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 16 / 60 | Data Modeling

CONCEPT AND DEFINITIONS

Reduction of an ER Diagram to Tables

Simple definition

ER-to-table mapping turns the conceptual diagram into relational tables while preserving entities, keys, and relationships.

Formal / technical definition

A mapping procedure creates relations for regular entity types, folds 1:1 or 1:N relationships into a suitable relation when appropriate, and creates a separate relation for M:N relationships and multivalued attributes. Constraints must be represented with keys and foreign keys where possible.

Notation and how to read it

Strong entity -> table; composite attribute -> components; multivalued attribute -> new table; 1:N -> FK on N-side; M:N -> junction table

How it works

- Read the requirement and identify entity types, attributes, and relationship meaning.
- Mark keys and minimum/maximum participation before drawing the final diagram.
- Check whether the diagram captures optionality, relationship attributes, and degree.

Quick map / trace / syntax

```
Student 1 --- N Enrollment N --- 1 Course
STUDENT(SID PK) ENROLL(SID FK, CID FK, Grade) COURSE(CID PK)
```

QUICK RECALL

Map keys first, then relationships, then special attributes; verify each relationship can be reconstructed by joins.

TOPIC 16 / 60 | Data Modeling

CODE EXAMPLE AND EXAM HELP

Reduction of an ER Diagram to Tables

Worked code example and output

```
STUDENT(SID PK, Name); COURSE(CID PK, Title); ENROLL(SID FK, CID FK, Grade, PK(SID,CID)).
ENROLL represents M:N and stores relationship attribute Grade.
```

Useful distinction / limitation

A foreign key can represent only relationships compatible with the chosen table design; preserve total participation and uniqueness constraints explicitly where needed.

Common mistakes to avoid

- An M:N relationship usually needs a separate relation.
- Put a 1:N foreign key on the N-side; preserve relationship attributes.

Quick viva and follow-up cues

- Core answer: Map keys first, then relationships, then special attributes; verify each relationship can be reconstructed by joins.
- Likely follow-up: A foreign key can represent only relationships compatible with the chosen table design; preserve total participation and uniqueness constraints explicitly where needed.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Map keys first, then relationships, then special attributes; verify each relationship can be reconstructed by joins.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 17 / 60 | Data Modeling

CONCEPT AND DEFINITIONS

Extended ER (EER) Model

Simple definition

EER adds richer ideas to ER diagrams, especially subclasses, inheritance, and constraints on specialization.

Formal / technical definition

The Extended ER model supplements entity, relationship, and attribute constructs with superclass/subclass structures, specialization/generalization, inheritance, and specialization constraints such as disjoint/overlapping and total/partial coverage.

Notation and how to read it

Disjoint/overlapping = subtype overlap rule; total/partial = subtype coverage rule

How it works

- Read the requirement and identify entity types, attributes, and relationship meaning.
- Mark keys and minimum/maximum participation before drawing the final diagram.
- Check whether the diagram captures optionality, relationship attributes, and degree.

Quick map / trace / syntax

```
PERSON
ISA (total, disjoint)
/ \
STUDENT EMPLOYEE
```

QUICK RECALL

Name the superclass, subclasses, inherited attributes, and both specialization constraints.

TOPIC 17 / 60 | Data Modeling

CODE EXAMPLE AND EXAM HELP

Extended ER (EER) Model

Worked code example and output

```
PERSON can specialize into STUDENT and EMPLOYEE. Disjoint means one person cannot be both
in this specialization; overlapping permits both. Total means every PERSON is in a
subtype; partial permits neither.
```

Useful distinction / limitation

Total/partial and disjoint/overlapping are separate dimensions: a specialization can be total and overlapping.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Name the superclass, subclasses, inherited attributes, and both specialization constraints.
- Likely follow-up: Total/partial and disjoint/overlapping are separate dimensions: a specialization can be total and overlapping.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Name the superclass, subclasses, inherited attributes, and both specialization constraints.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 18 / 60 | Data Modeling

CONCEPT AND DEFINITIONS

Relationship of Higher Degree

Simple definition

A higher-degree relationship connects three or more entity types at once; the relationship meaning depends on the whole combination.

Formal / technical definition

The degree of a relationship is the number of participating entity types. A ternary relationship has degree three. It cannot always be replaced by several binary relationships without changing semantics or losing combination-level constraints.

Notation and how to read it

degree 2 = binary; degree 3 = ternary; degree n = n-ary

How it works

- Read the requirement and identify entity types, attributes, and relationship meaning.
- Mark keys and minimum/maximum participation before drawing the final diagram.
- Check whether the diagram captures optionality, relationship attributes, and degree.

Quick map / trace / syntax

```
SUPPLIER --\
PART ----- SUPPLIES ----- PROJECT
meaning belongs to one supplier-part-project combination
```

QUICK RECALL

State the participating entity types and give one fact that is true only for the complete tuple.

TOPIC 18 / 60 | Data Modeling

CODE EXAMPLE AND EXAM HELP

Relationship of Higher Degree

Worked code example and output

```
SUPPLIER supplies PART to PROJECT. The fact may depend on this exact supplier-part-project triple; separate binary links might incorrectly imply every supplier supplies every linked part to every linked project.
```

Useful distinction / limitation

Do not decompose an n-ary relationship just because the diagram is visually complex; first check whether the binary version preserves meaning.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: State the participating entity types and give one fact that is true only for the complete tuple.
- Likely follow-up: Do not decompose an n-ary relationship just because the diagram is visually complex; first check whether the binary version preserves meaning.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: State the participating entity types and give one fact that is true only for the complete tuple.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 19 / 60 | Relational Data Model

CONCEPT AND DEFINITIONS

Relational Data Model Concepts

Simple definition

The relational model stores data in relations, usually shown as tables with rows and columns.

Formal / technical definition

A relation schema names attributes and their domains. A relation instance is a set of tuples over those domains. In the mathematical model, tuples are unordered and duplicates are absent; SQL tables may permit duplicates unless constrained.

Notation and how to read it

Relation = table-like set; tuple = row; attribute = column; domain = allowed value set; degree = number of attributes; cardinality = number of tuples

How it works

- Name the relation, attributes, domains, and applicable constraint.
- Test the rule on valid and invalid row examples.
- Explain the effect on inserts, updates, deletes, or query results.

Quick map / trace / syntax

```
Student(SID, Name, Dept)
(17, Asha, CSE)
(18, Ravi, ECE)
```

QUICK RECALL

Define relation, tuple, attribute, domain, degree, and cardinality with one small example.

TOPIC 19 / 60 | Relational Data Model

CODE EXAMPLE AND EXAM HELP

Relational Data Model Concepts

Worked code example and output

```
Student(SID, Name, Dept) has attributes SID, Name, Dept. A tuple (17, 'Asha', 'CSE') is one row; SID's domain may be integers.
```

Useful distinction / limitation

Do not confuse relation cardinality (row count) with degree (column count). The mathematical relation is a set, while SQL's bag behavior can retain duplicates.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Define relation, tuple, attribute, domain, degree, and cardinality with one small example.
- Likely follow-up: Do not confuse relation cardinality (row count) with degree (column count). The mathematical relation is a set, while SQL's bag behavior can retain duplicates.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Define relation, tuple, attribute, domain, degree, and cardinality with one small example.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 20 / 60 | Relational Data Model

CONCEPT AND DEFINITIONS

Integrity Constraints

Simple definition

Integrity constraints are rules that keep stored data valid and consistent.

Formal / technical definition

A database constraint is a predicate that every legal database state must satisfy. Relational constraints include domain, key, entity-integrity, and referential-integrity constraints; they may be declared in DDL or enforced by application logic.

Notation and how to read it

Valid state D iff every declared constraint evaluates TRUE (with SQL NULL semantics considered)

How it works

- Name the relation, attributes, domains, and applicable constraint.
- Test the rule on valid and invalid row examples.
- Explain the effect on inserts, updates, deletes, or query results.

Quick map / trace / syntax

```
User update -> constraint check -> accept if valid / reject or apply declared action
```

QUICK RECALL

Name the rule, identify the affected attributes/relations, and state what happens when it is violated.

TOPIC 20 / 60 | Relational Data Model

CODE EXAMPLE AND EXAM HELP

Integrity Constraints

Worked code example and output

```
A grade may be constrained to a known range; every Student SID must be unique; every Enrollment SID must refer to an existing Student.
```

Useful distinction / limitation

A constraint describes a rule for all permitted states, not just a check on one sample row.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Name the rule, identify the affected attributes/relations, and state what happens when it is violated.
- Likely follow-up: A constraint describes a rule for all permitted states, not just a check on one sample row.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Name the rule, identify the affected attributes/relations, and state what happens when it is violated.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 21 / 60 | Relational Data Model

CONCEPT AND DEFINITIONS

Entity Integrity

Simple definition

Entity integrity says a primary-key value must exist for every row, so each row can be identified.

Formal / technical definition

The entity-integrity rule requires every attribute of a relation's primary key to be non-NULL. A candidate/primary key also uniquely identifies tuples under the relational model.

Notation and how to read it

PRIMARY KEY = UNIQUE + NOT NULL (for all key columns)

How it works

- Name the relation, attributes, domains, and applicable constraint.
- Test the rule on valid and invalid row examples.
- Explain the effect on inserts, updates, deletes, or query results.

Quick map / trace / syntax

```
Student row -> SID present -> unique identity
NULL SID -> rejected; duplicate SID -> rejected
```

QUICK RECALL

For a composite key, test uniqueness of the combination and non-nullness of each component.

TOPIC 21 / 60 | Relational Data Model

CODE EXAMPLE AND EXAM HELP

Entity Integrity

Worked code example and output

```
Student(SID PRIMARY KEY, Name): SID cannot be missing and no two rows can have the same SID.
```

Useful distinction / limitation

A composite primary key requires every component to be non-NULL; an individual component need not be unique by itself.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: For a composite key, test uniqueness of the combination and non-nullness of each component.
- Likely follow-up: A composite primary key requires every component to be non-NULL; an individual component need not be unique by itself.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: For a composite key, test uniqueness of the combination and non-nullness of each component.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 22 / 60 | Relational Data Model

CONCEPT AND DEFINITIONS

Referential Integrity

Simple definition

Referential integrity prevents a reference from pointing to a row that does not exist.

Formal / technical definition

For a foreign key F in relation R referencing candidate key K of relation S, each non-NULL F value must match a K value in S, subject to the DBMS's MATCH and NULL rules.

Notation and how to read it

FOREIGN KEY (SID) REFERENCES Student(SID); actions may include RESTRICT/NO ACTION, CASCADE, SET NULL, SET DEFAULT (vendor support varies)

How it works

- Name the relation, attributes, domains, and applicable constraint.
- Test the rule on valid and invalid row examples.
- Explain the effect on inserts, updates, deletes, or query results.

Quick map / trace / syntax

```
ENROLL.SID -> STUDENT.SID
insert/update reference -> matching parent key required
```

QUICK RECALL

Explain the child table, referenced parent key, and outcome for a missing parent row.

TOPIC 22 / 60 | Relational Data Model

CODE EXAMPLE AND EXAM HELP

Referential Integrity

Worked code example and output

```
Enrollment(SID, CID) references Student(SID) and Course(CID). An enrollment with SID=99 is rejected if Student 99 does not exist.
```

Useful distinction / limitation

Foreign keys may allow NULL unless declared NOT NULL; a nullable foreign key represents a missing/optional relationship, not a dangling reference.

Common mistakes to avoid

- A nullable foreign key is not a dangling reference.
- Identify the referenced parent key and the configured delete/update action.

Quick viva and follow-up cues

- Core answer: Explain the child table, referenced parent key, and outcome for a missing parent row.
- Likely follow-up: Foreign keys may allow NULL unless declared NOT NULL; a nullable foreign key represents a missing/optional relationship, not a dangling reference.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Explain the child table, referenced parent key, and outcome for a missing parent row.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 23 / 60 | Relational Data Model

CONCEPT AND DEFINITIONS

Key Constraints

Simple definition

Key constraints require chosen attribute values, or combinations of them, to identify rows uniquely.

Formal / technical definition

A key constraint requires that no two distinct tuples agree on all attributes of a key set. Candidate keys are minimal unique attribute sets; primary and alternate keys are schema choices based on these constraints.

Notation and how to read it

For key K: if t1[K]=t2[K], then t1=t2

How it works

- Name the relation, attributes, domains, and applicable constraint.
- Test the rule on valid and invalid row examples.
- Explain the effect on inserts, updates, deletes, or query results.

Quick map / trace / syntax

```
(SID=17,CID=CS101) occurs once
(SID=17,CID=MA102) is a different key value
```

QUICK RECALL

State the candidate key and explain why each proper subset fails to identify a row.

TOPIC 23 / 60 | Relational Data Model

CODE EXAMPLE AND EXAM HELP

Key Constraints

Worked code example and output

```
Enrollment(SID, CID, Grade) may use composite key (SID, CID) if a student enrolls only once in each course.
```

Useful distinction / limitation

A key does not automatically enforce a business rule that has not been stated; verify assumptions such as one enrollment per term.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: State the candidate key and explain why each proper subset fails to identify a row.
- Likely follow-up: A key does not automatically enforce a business rule that has not been stated; verify assumptions such as one enrollment per term.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: State the candidate key and explain why each proper subset fails to identify a row.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 24 / 60 | Relational Data Model

CONCEPT AND DEFINITIONS

Domain Constraints

Simple definition

A domain constraint limits a column to values of the right kind and range.

Formal / technical definition

Each attribute is associated with a domain of atomic values. Type declarations, NOT NULL, CHECK, and enumerated constraints can restrict values, though detailed support varies by DBMS.

Notation and how to read it

attribute A in tuple t -> t[A] must belong to domain(A) and satisfy declared predicates

How it works

- Name the relation, attributes, domains, and applicable constraint.
- Test the rule on valid and invalid row examples.
- Explain the effect on inserts, updates, deletes, or query results.

Quick map / trace / syntax

```
Age = -4 -> violates CHECK
Age = 20 -> inside allowed domain
```

QUICK RECALL

Identify type, nullability, and any allowed-range or allowed-set condition.

TOPIC 24 / 60 | Relational Data Model

CODE EXAMPLE AND EXAM HELP

Domain Constraints

Worked code example and output

```
Age INTEGER CHECK (Age BETWEEN 0 AND 130); Grade CHAR(2) CHECK (Grade IN
('A','B','C','D','F')).
```

Useful distinction / limitation

A data type alone may be too broad for a business rule; use an additional constraint when the domain requires it.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Identify type, nullability, and any allowed-range or allowed-set condition.
- Likely follow-up: A data type alone may be too broad for a business rule; use an additional constraint when the domain requires it.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Identify type, nullability, and any allowed-range or allowed-set condition.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 25 / 60 | Relational Data Model

CONCEPT AND DEFINITIONS

Relational Algebra

Simple definition

Relational algebra is a step-by-step language for querying relations. Each operation produces another relation.

Formal / technical definition

Relational algebra is a procedural formal query language whose operators take relations as input and return relations. Core operations include selection, projection, union, difference, Cartesian product, and rename; derived operators include joins and intersection.

Notation and how to read it

sigma_condition(R)=selection; pi_attrs(R)=projection; R union S; R-S; R x S; rho_newname(R); join R bowtie_condition S

How it works

- Name each input relation and required result columns.
- Apply selection/projection/set or join operators in a checkable order.
- Verify schemas, tuple conditions, and duplicate assumptions at every step.

Quick map / trace / syntax

```
Student -> sigma Dept='CSE' -> matching rows -> pi Name -> names only
```

QUICK RECALL

Write the intermediate relation after each operator and check schema compatibility.

TOPIC 25 / 60 | Relational Data Model

CODE EXAMPLE AND EXAM HELP

Relational Algebra

Worked code example and output

```
Given Student(SID, Name, Dept), find CSE names: pi_Name(sigma_Dept='CSE'(Student)).
Selection filters rows first; projection keeps chosen columns.
```

Useful distinction / limitation

Union and difference require union-compatible relations; projection may remove duplicates in classical set-based algebra.

Common mistakes to avoid

- Union and difference need union-compatible schemas.
- Projection can remove duplicates in set-based relational algebra; SQL behavior differs.

Quick viva and follow-up cues

- Core answer: Write the intermediate relation after each operator and check schema compatibility.
- Likely follow-up: Union and difference require union-compatible relations; projection may remove duplicates in classical set-based algebra.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Write the intermediate relation after each operator and check schema compatibility.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 26 / 60 | Relational Data Model

CONCEPT AND DEFINITIONS

Relational Calculus

Simple definition

Relational calculus describes what result is wanted by stating logical conditions, rather than listing a sequence of table operations.

Formal / technical definition

Relational calculus is a declarative, predicate-based formal query language. Tuple and domain versions are equivalent in expressive power to relational algebra for safe relational queries (Codd's relational completeness result).

Notation and how to read it

calculus query = { result | predicate(result, database) }

'|' reads as such that

How it works

- Choose tuple variables or domain-value variables.
- Write the predicate that identifies matching database values.
- Project only the desired output and ensure the expression is safe.

Quick map / trace / syntax

```
Specify desired values + logical predicate -> set of satisfying results
```

QUICK RECALL

Emphasize declarative meaning and distinguish the result condition from an execution algorithm.

TOPIC 26 / 60 | Relational Data Model

CODE EXAMPLE AND EXAM HELP

Relational Calculus

Worked code example and output

```
Ask for names of students in CSE by describing tuples that satisfy Dept='CSE' and
selecting their Name; the DBMS can choose the evaluation plan.
```

Useful distinction / limitation

The calculus formulation states conditions, but safety/range restriction matters so results remain finite and domain-independent.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Emphasize declarative meaning and distinguish the result condition from an execution algorithm.
- Likely follow-up: The calculus formulation states conditions, but safety/range restriction matters so results remain finite and domain-independent.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Emphasize declarative meaning and distinguish the result condition from an execution algorithm.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 27 / 60 | Relational Data Model

CONCEPT AND DEFINITIONS

Tuple Relational Calculus

Simple definition

Tuple relational calculus uses tuple variables: each variable stands for a whole row.

Formal / technical definition

A tuple-calculus expression has the form {t | P(t)}, where t ranges over tuples and P is a first-order predicate involving relation membership, comparisons, and logical connectives/quantifiers.

Notation and how to read it

{ t.A1,...,t.An | P(t) }; exists u(P); for all u(P)

How it works

- Choose tuple variables or domain-value variables.
- Write the predicate that identifies matching database values.
- Project only the desired output and ensure the expression is safe.

Quick map / trace / syntax

```
t ranges over Student rows
filter condition t.Dept='CSE'
return t.Name
```

QUICK RECALL

Say what each tuple variable ranges over, state the predicate, and identify the projected result fields.

TOPIC 27 / 60 | Relational Data Model

CODE EXAMPLE AND EXAM HELP

Tuple Relational Calculus

Worked code example and output

```
{ t.Name | Student(t) AND t.Dept='CSE' } returns names of tuples t in Student whose Dept is CSE.
```

Useful distinction / limitation

A query should be range-restricted/safe. Avoid expressions whose satisfying values come from an unbounded outside domain.

Common mistakes to avoid

- Tuple variables stand for whole tuples, not individual column values.
- Keep expressions safe/range-restricted so results remain finite.

Quick viva and follow-up cues

- Core answer: Say what each tuple variable ranges over, state the predicate, and identify the projected result fields.
- Likely follow-up: A query should be range-restricted/safe. Avoid expressions whose satisfying values come from an unbounded outside domain.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Say what each tuple variable ranges over, state the predicate, and identify the projected result fields.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 28 / 60 | Relational Data Model

CONCEPT AND DEFINITIONS

Domain Relational Calculus

Simple definition

Domain relational calculus uses variables for individual column values rather than whole rows.

Formal / technical definition

A domain-calculus expression has the form {<x1,...,xn> | P(x1,...,xn)}, where variables range over attribute domains and P is a predicate specifying valid output tuples.

Notation and how to read it

{<x1,...,xn> | P(x1,...,xn)}; exists/for all quantify domain variables

How it works

- Choose tuple variables or domain-value variables.
- Write the predicate that identifies matching database values.
- Project only the desired output and ensure the expression is safe.

Quick map / trace / syntax

```
sid, n, d are column-value variables
Student(sid,n,d) AND d='CSE' -> output <n>
```

QUICK RECALL

Show the output variables and the existentially quantified variables separately.

TOPIC 28 / 60 | Relational Data Model

CODE EXAMPLE AND EXAM HELP

Domain Relational Calculus

Worked code example and output

```
{<n> | exists sid, d (Student(sid,n,d) AND d='CSE')} returns names n of students whose department is CSE.
```

Useful distinction / limitation

Tuple calculus binds a whole tuple variable; domain calculus binds individual attribute values. Both need safe range restrictions.

Common mistakes to avoid

- Domain variables stand for individual attribute values.
- Separate returned variables from existentially quantified variables.

Quick viva and follow-up cues

- Core answer: Show the output variables and the existentially quantified variables separately.
- Likely follow-up: Tuple calculus binds a whole tuple variable; domain calculus binds individual attribute values. Both need safe range restrictions.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Show the output variables and the existentially quantified variables separately.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

MODULE-I COMPLETE

Pause. Review. Then continue.

MODULE-I established database architecture, ER design, relational constraints, and formal query foundations.

Before you begin the next module

- Close the notes and explain one key idea from memory.
- Redo one ER mapping or SQL trace without looking at the result.
- Preview the next roadmap and list the concepts you will need.

NEXT MODULE STARTS NEXT

MODULE-II - Introduction on SQL and Data Base Design & Normalization

QUICK RECALL

Small, repeated practice beats rereading a full chapter once.

MODULE-II - what to study and in what order

Module goal

Use SQL to create and query data, then design relations that preserve facts with fewer anomalies.

Syllabus map - exact topic order

- Introduction on SQL: Characteristics of SQL; Advantage of SQL; SQL Data Types and Literals; Types of SQL Commands; SQL Operators; Tables; Views; Indexes; INSERT Operation; UPDATE Operation; DELETE Operation; Queries; Subqueries; Aggregate Functions; Joins; UNION; INTERSECTION; MINUS; Cursors; Triggers.
- Data Base Design & Normalization: Functional Dependencies; Normal Forms; First Normal Form (1NF); Second Normal Form (2NF); Third Normal Form (3NF); Boyce-Codd Normal Form (BCNF); Inclusion Dependence; Lossless Join Decompositions; Normalization Using Functional Dependencies; Normalization Using Multivalued Dependencies (MVD); Normalization Using Join Dependencies (JD); Alternative Approaches to Database Design.

Recommended study order

- Learn SQL command groups and types, then DML/query clauses and set operations.
- Trace aggregates, joins, cursors, and triggers with small examples.
- Use functional dependencies and normal-form definitions to evaluate decompositions.

QUICK RECALL

Before finishing, compute a key closure, test a normal form, and check decomposition losslessness.

TOPIC 29 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

Characteristics of SQL

Simple definition

SQL is the common language used to define relational database objects and work with their data.

Formal / technical definition

Structured Query Language is a declarative language for relational schema definition, data retrieval/manipulation, access control, and transaction control. Standards define a core; products implement dialects and extensions.

Notation and how to read it

SQL: declarative; set-oriented; supports schema, queries, updates, constraints, views, authorization and transaction statements

How it works

- Identify the statement goal: define, retrieve/change, authorize, or control a transaction.
- Choose valid syntax and check types, constraints, and privileges.
- Explain what changes and what result/status is returned.

Quick map / trace / syntax

```
SQL statement -> parse -> validate -> optimize -> execute -> result/status
```

QUICK RECALL

State that SQL is declarative and identify its major uses.

TOPIC 29 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Characteristics of SQL

Worked code example and output

`SELECT Name FROM Student WHERE Dept='CSE';` describes the desired rows; the optimizer chooses an execution strategy.

Useful distinction / limitation

SQL syntax is not identical across all vendors; note a dialect when using vendor-specific features.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: State that SQL is declarative and identify its major uses.
- Likely follow-up: SQL syntax is not identical across all vendors; note a dialect when using vendor-specific features.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: State that SQL is declarative and identify its major uses.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 30 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

Advantage of SQL

Simple definition

SQL gives users a standard, concise way to ask questions about relational data without writing low-level file-processing code.

Formal / technical definition

SQL supports high-level set operations, data definition and manipulation, constraint declaration, authorization, and transaction commands. It is widely implemented and can be embedded in application programs.

Notation and how to read it

Advantages: high-level queries; set processing; portability of core concepts; integration with drivers/tools; permissions and constraints

How it works

- Identify the statement goal: define, retrieve/change, authorize, or control a transaction.
- Choose valid syntax and check types, constraints, and privileges.
- Explain what changes and what result/status is returned.

Quick map / trace / syntax

```
One declarative statement -> DBMS processes all qualifying rows
```

QUICK RECALL

Give two benefits and one realistic limitation of portability.

TOPIC 30 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Advantage of SQL

Worked code example and output

```
One statement updates all overdue invoices: UPDATE Invoice SET Status='Late' WHERE DueDate
< CURRENT_DATE AND Status='Open';
```

Useful distinction / limitation

Portability is strongest for standard features; vendor-specific types, functions, and procedural syntax may require changes.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Give two benefits and one realistic limitation of portability.
- Likely follow-up: Portability is strongest for standard features; vendor-specific types, functions, and procedural syntax may require changes.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Give two benefits and one realistic limitation of portability.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 31 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

SQL Data Types and Literals

Simple definition

A column's type describes what values it stores. A literal is a value written directly in a SQL statement.

Formal / technical definition

SQL types include numeric, character, boolean (support varies), date/time, and binary families. Literals use type-appropriate syntax; NULL denotes an unknown/missing value and is not equal to ordinary values.

Notation and how to read it

numeric: 17, 8.25; character: 'Asha'; date: DATE '2025-07-01'; NULL; boolean literals vary by product

How it works

- Identify the statement goal: define, retrieve/change, authorize, or control a transaction.
- Choose valid syntax and check types, constraints, and privileges.
- Explain what changes and what result/status is returned.

Quick map / trace / syntax

```
literal -> type/domain check -> stored value
NULL test: Joined IS NULL (not Joined = NULL)
```

QUICK RECALL

Match literal syntax and data type; use IS NULL/IS NOT NULL for null checks.

TOPIC 31 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

SQL Data Types and Literals

Worked code example and output

```
CREATE TABLE Student(SID INTEGER, Name VARCHAR(60), GPA DECIMAL(3,2), Joined DATE);
INSERT INTO Student VALUES (17, 'Asha', 8.25, DATE '2025-07-01');
```

Useful distinction / limitation

A quoted number such as '17' is text in many contexts. NULL means unknown, so comparisons use three-valued logic.

Common mistakes to avoid

- Use IS NULL instead of = NULL.
- Quoted numbers are character literals; match literal types to column types.

Quick viva and follow-up cues

- Core answer: Match literal syntax and data type; use IS NULL/IS NOT NULL for null checks.
- Likely follow-up: A quoted number such as '17' is text in many contexts. NULL means unknown, so comparisons use three-valued logic.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Match literal syntax and data type; use IS NULL/IS NOT NULL for null checks.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 32 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

Types of SQL Commands

Simple definition

SQL commands are often grouped by whether they define structures, manipulate data, control permissions, or manage transactions.

Formal / technical definition

Common teaching categories are DDL (CREATE, ALTER, DROP), DML (INSERT, UPDATE, DELETE), DQL (SELECT, often grouped with DML), DCL (GRANT, REVOKE), and TCL (COMMIT, ROLLBACK, SAVEPOINT). Classifications can vary among texts/products.

Notation and how to read it

DDL = schema; DML = row changes; DQL = query; DCL = access; TCL = transaction boundary

How it works

- Identify the statement goal: define, retrieve/change, authorize, or control a transaction.
- Choose valid syntax and check types, constraints, and privileges.
- Explain what changes and what result/status is returned.

Quick map / trace / syntax

```
SQL command -> category depends on its purpose, not merely whether it starts a transaction
```

QUICK RECALL

For each command, name the purpose and a representative statement.

TOPIC 32 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Types of SQL Commands

Worked code example and output

```
CREATE TABLE Student(...); INSERT INTO Student...; SELECT...; GRANT SELECT...; COMMIT;
```

Useful distinction / limitation

Some textbooks classify SELECT as DML; mention this convention if a question expects a specific list.

Common mistakes to avoid

- Some texts classify SELECT as DQL and others include it under DML.
- Distinguish schema definition, row change, authorization, and transaction commands.

Quick viva and follow-up cues

- Core answer: For each command, name the purpose and a representative statement.
- Likely follow-up: Some textbooks classify SELECT as DML; mention this convention if a question expects a specific list.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: For each command, name the purpose and a representative statement.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 33 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

SQL Operators

Simple definition

Operators compare or combine values so a query can calculate, filter, or match rows.

Formal / technical definition

SQL expressions support arithmetic, comparison, logical, predicate, and set operators. NULL introduces UNKNOWN under three-valued logic; WHERE retains rows whose condition is TRUE.

Notation and how to read it

=, <>, <, <=, >, >=; AND, OR, NOT; +,-,*,/; IN, BETWEEN, LIKE, IS NULL; concatenation varies

How it works

- Identify the statement goal: define, retrieve/change, authorize, or control a transaction.
- Choose valid syntax and check types, constraints, and privileges.
- Explain what changes and what result/status is returned.

Quick map / trace / syntax

```
predicate TRUE -> row qualifies; FALSE/UNKNOWN -> row excluded by WHERE
```

QUICK RECALL

Trace operator precedence and test a condition with NULL as well as ordinary values.

TOPIC 33 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

SQL Operators

Worked code example and output

```
SELECT SID FROM Student WHERE Dept='CSE' AND GPA >= 8.0;
SELECT Name FROM Student WHERE Name LIKE 'A%';
```

Useful distinction / limitation

Use parentheses when mixing AND and OR. NULL comparisons are not ordinary equality comparisons.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Trace operator precedence and test a condition with NULL as well as ordinary values.
- Likely follow-up: Use parentheses when mixing AND and OR. NULL comparisons are not ordinary equality comparisons.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Trace operator precedence and test a condition with NULL as well as ordinary values.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 34 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

Tables

Simple definition

A table is a named structure that stores rows with the same set of columns.

Formal / technical definition

A base table is a relation-like database object with named columns, declared types, and optional constraints. SQL tables may allow duplicate rows unless keys or other uniqueness rules prevent them.

Notation and how to read it

Table = columns (schema) + rows (current contents); constraints restrict legal row states

How it works

- Identify the statement goal: define, retrieve/change, authorize, or control a transaction.
- Choose valid syntax and check types, constraints, and privileges.
- Explain what changes and what result/status is returned.

Quick map / trace / syntax

```
Course(CID, Title, Credits)
PK: CID; NOT NULL: Title; CHECK: Credits > 0
```

QUICK RECALL

Separate table definition from the rows currently stored in it.

TOPIC 34 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Tables

Worked code example and output

```
CREATE TABLE Course (CID VARCHAR(10) PRIMARY KEY, Title VARCHAR(80) NOT NULL, Credits
INTEGER CHECK (Credits > 0));
```

Useful distinction / limitation

A relational table's row order is not guaranteed. Use ORDER BY when a particular output order is required.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Separate table definition from the rows currently stored in it.
- Likely follow-up: A relational table's row order is not guaranteed. Use ORDER BY when a particular output order is required.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Separate table definition from the rows currently stored in it.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

Views

Simple definition

A view is a named query that presents selected or combined data like a virtual table.

Formal / technical definition

A conventional view stores a query definition in the catalog; its contents are derived when referenced. A materialized view stores results and requires refresh semantics. Updatability depends on the view and DBMS.

Notation and how to read it

view name -> stored SELECT definition -> derived result when queried

How it works

- Identify the statement goal: define, retrieve/change, authorize, or control a transaction.
- Choose valid syntax and check types, constraints, and privileges.
- Explain what changes and what result/status is returned.

Quick map / trace / syntax

```
Base tables -> SELECT view definition -> view rows
view can hide columns/rows from a user
```

QUICK RECALL

Explain the saved query, what the view exposes, and whether results are stored or derived.

TOPIC 35 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Views

Worked code example and output

```
CREATE VIEW CSE_Students AS SELECT SID, Name FROM Student WHERE Dept='CSE';
SELECT * FROM CSE_Students;
```

Useful distinction / limitation

A normal view is not necessarily a copied table. A view is not automatically a complete security solution; privileges must be configured.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Explain the saved query, what the view exposes, and whether results are stored or derived.
- Likely follow-up: A normal view is not necessarily a copied table. A view is not automatically a complete security solution; privileges must be configured.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Explain the saved query, what the view exposes, and whether results are stored or derived.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 36 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

Indexes

Simple definition

An index is an extra access structure that can help the DBMS find rows faster, usually at the cost of storage and slower writes.

Formal / technical definition

An index maps search-key values to row locations or keys using a structure such as a B-tree or hash. The optimizer may use it for filtering, ordering, or joins; the DBMS maintains it as table data changes.

Notation and how to read it

search key -> index entry -> candidate row/page; unique index also enforces uniqueness where supported

How it works

- Identify the statement goal: define, retrieve/change, authorize, or control a transaction.
- Choose valid syntax and check types, constraints, and privileges.
- Explain what changes and what result/status is returned.

Quick map / trace / syntax

```
Without index: scan rows
With usable index: navigate index -> fetch matching rows
```

QUICK RECALL

State the lookup benefit and the storage/write-maintenance trade-off.

TOPIC 36 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Indexes

Worked code example and output

```
CREATE INDEX idx_student_dept ON Student(Dept);
SELECT SID, Name FROM Student WHERE Dept='CSE';
```

Useful distinction / limitation

Indexes consume space and add INSERT/UPDATE/DELETE maintenance. The optimizer may ignore an index if a scan is cheaper.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: State the lookup benefit and the storage/write-maintenance trade-off.
- Likely follow-up: Indexes consume space and add INSERT/UPDATE/DELETE maintenance. The optimizer may ignore an index if a scan is cheaper.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: State the lookup benefit and the storage/write-maintenance trade-off.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 37 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

INSERT Operation

Simple definition

INSERT adds one or more rows to a table.

Formal / technical definition

An INSERT statement supplies values for named columns or obtains rows from a query. Values must satisfy types, defaults, keys, and other declared constraints.

Notation and how to read it

INSERT INTO table (columns) VALUES (values);

INSERT INTO table(columns) SELECT ...;

How it works

- Identify the target table and the exact rows/columns affected.
- Check values, WHERE condition, and key/domain/referential constraints.
- Predict the resulting table and consider transaction/recovery needs.

Quick map / trace / syntax

```
column list -> matching values -> type/constraint checks -> row added
```

QUICK RECALL

Count columns and values, check primary/foreign keys, and use a column list for reliable inserts.

TOPIC 37 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

INSERT Operation

Worked code example and output

```
INSERT INTO Student (SID, Name, Dept, GPA)
VALUES (17, 'Asha', 'CSE', 8.25);

INSERT INTO CSE_List(SID, Name)
SELECT SID, Name FROM Student WHERE Dept='CSE';
```

Useful distinction / limitation

Omitting a column uses its default or NULL if permitted; without a column list, values must match the table's column order.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Count columns and values, check primary/foreign keys, and use a column list for reliable inserts.
- Likely follow-up: Omitting a column uses its default or NULL if permitted; without a column list, values must match the table's column order.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Count columns and values, check primary/foreign keys, and use a column list for reliable inserts.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 38 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

UPDATE Operation

Simple definition

UPDATE changes values in existing rows that satisfy a condition.

Formal / technical definition

UPDATE assigns expressions to one or more columns of rows selected by its search condition. Constraints are checked against the resulting state; omitted WHERE targets every row.

Notation and how to read it

UPDATE table SET column=expression [, ...] WHERE condition;

How it works

- Identify the target table and the exact rows/columns affected.
- Check values, WHERE condition, and key/domain/referential constraints.
- Predict the resulting table and consider transaction/recovery needs.

Quick map / trace / syntax

```
evaluate WHERE -> identify rows -> compute assignments -> validate -> write changes
```

QUICK RECALL

Before the update, state which rows match; then compute their new values.

TOPIC 38 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

UPDATE Operation

Worked code example and output

```
UPDATE Student
SET GPA = GPA + 0.10
WHERE SID = 17;
```

Useful distinction / limitation

A missing WHERE clause updates all rows. Previewing the predicate with SELECT can help verify the target set.

Common mistakes to avoid

- Without WHERE, UPDATE affects every row.
- Preview the predicate and calculate changed values before writing the statement.

Quick viva and follow-up cues

- Core answer: Before the update, state which rows match; then compute their new values.
- Likely follow-up: A missing WHERE clause updates all rows. Previewing the predicate with SELECT can help verify the target set.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Before the update, state which rows match; then compute their new values.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 39 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

DELETE Operation

Simple definition

DELETE removes selected rows. The table definition remains unless a separate DDL command removes it.

Formal / technical definition

A searched DELETE removes all rows satisfying its predicate. Foreign-key actions or restrictions may affect whether deletion is allowed. Without WHERE, all rows are removed while the table remains.

Notation and how to read it

DELETE FROM table WHERE condition;

How it works

- Identify the target table and the exact rows/columns affected.
- Check values, WHERE condition, and key/domain/referential constraints.
- Predict the resulting table and consider transaction/recovery needs.

Quick map / trace / syntax

```
predicate identifies rows -> referential actions/constraints checked -> qualifying rows removed
```

QUICK RECALL

Contrast DELETE with DROP and explain the exact effect of the WHERE clause.

TOPIC 39 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

DELETE Operation

Worked code example and output

```
DELETE FROM Student WHERE SID = 17;
-- DELETE FROM Student; removes every row
```

Useful distinction / limitation

DELETE removes rows; DROP TABLE removes the table object. A WHERE omission is a common high-impact error.

Common mistakes to avoid

- DELETE removes rows; DROP removes a table object.
- Without WHERE, DELETE removes all rows (subject to constraints).

Quick viva and follow-up cues

- Core answer: Contrast DELETE with DROP and explain the exact effect of the WHERE clause.
- Likely follow-up: DELETE removes rows; DROP TABLE removes the table object. A WHERE omission is a common high-impact error.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Contrast DELETE with DROP and explain the exact effect of the WHERE clause.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 40 / 60 | Introduction on SQL CONCEPT AND DEFINITIONS

Queries

Simple definition

A query asks the database to return rows or calculated values that match specified conditions.

Formal / technical definition

A SELECT query forms a result from table expressions, filters rows, groups/aggregates where requested, filters groups, and projects/renames output columns. ORDER BY controls final presentation order.

Notation and how to read it

Logical query order: FROM/JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> DISTINCT -> ORDER BY -> FETCH/LIMIT (dialect varies)

How it works

- Identify source tables and join conditions.
- Apply WHERE, grouping, HAVING, output columns, and ordering in logical order.
- Test the query on a small sample and check duplicates/NULL behavior.

Quick map / trace / syntax

```
source rows -> filter -> groups -> group filter -> output columns -> sort
```

QUICK RECALL

Trace each clause in logical order and distinguish WHERE (rows) from HAVING (groups).

TOPIC 40 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Queries

Worked code example and output

```
SELECT Dept, COUNT(*) AS StudentCount
FROM Student
WHERE GPA >= 6.0
GROUP BY Dept
HAVING COUNT(*) >= 2
ORDER BY StudentCount DESC;
```

Useful distinction / limitation

Logical processing order helps explain aliases and aggregates, but the physical execution plan may use a different order.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Trace each clause in logical order and distinguish WHERE (rows) from HAVING (groups).
- Likely follow-up: Logical processing order helps explain aliases and aggregates, but the physical execution plan may use a different order.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Trace each clause in logical order and distinguish WHERE (rows) from HAVING (groups).

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 41 / 60 | Introduction on SQL CONCEPT AND DEFINITIONS

Subqueries

Simple definition

A subquery is a query placed inside another SQL statement; its result helps the outer query decide what to return or change.

Formal / technical definition

A subquery may be scalar, row-valued, or table-valued and may be correlated with the outer query. Common predicates include IN, EXISTS, comparison operators, and quantified ANY/ALL, subject to DBMS syntax.

Notation and how to read it

outer query; nested SELECT; correlated reference s.SID uses current outer row

How it works

- Separate inner and outer query scopes.
- Identify whether the subquery returns a scalar, set, or row source.
- Trace one outer row when correlated; check empty results and NULL.

Quick map / trace / syntax

```
For each candidate Student row -> test matching Enrollment -> EXISTS true/false -> keep or discard
```

QUICK RECALL

Label inner and outer queries and identify whether the subquery is correlated.

TOPIC 41 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Subqueries

Worked code example and output

```
SELECT Name FROM Student s
WHERE EXISTS (SELECT 1 FROM Enrollment e WHERE e.SID=s.SID);
```

Useful distinction / limitation

NOT IN can behave unexpectedly if the subquery returns NULL. NOT EXISTS is often safer for anti-matching logic.

Common mistakes to avoid

- A correlated subquery refers to a column of the current outer row.
- NOT IN with NULL can yield UNKNOWN; consider NOT EXISTS.

Quick viva and follow-up cues

- Core answer: Label inner and outer queries and identify whether the subquery is correlated.
- Likely follow-up: NOT IN can behave unexpectedly if the subquery returns NULL. NOT EXISTS is often safer for anti-matching logic.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Label inner and outer queries and identify whether the subquery is correlated.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 42 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

Aggregate Functions

Simple definition

Aggregate functions summarize many rows into one value, such as a count, average, smallest value, or largest value.

Formal / technical definition

Common aggregates include COUNT, SUM, AVG, MIN, and MAX. Except COUNT(*), most standard aggregates ignore NULL inputs. GROUP BY creates one result per group; HAVING filters groups.

Notation and how to read it

COUNT(*) counts rows; COUNT(col) counts non-NULL values; SUM/AVG/MIN/MAX summarize values

How it works

- Choose the grouping columns and aggregate expression.
- Filter rows before grouping and groups after grouping.
- Check COUNT(*) vs COUNT(column) and how NULL is handled.

Quick map / trace / syntax

```
rows -> GROUP BY Dept -> aggregate per group -> HAVING removes groups
```

QUICK RECALL

Show grouping columns, aggregate input, and whether NULL rows contribute.

TOPIC 42 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Aggregate Functions

Worked code example and output

```
SELECT Dept, AVG(GPA) AS MeanGPA
FROM Student
GROUP BY Dept
HAVING COUNT(*) >= 2;
```

Useful distinction / limitation

COUNT(*) and COUNT(column) differ when the column contains NULL. Do not select a nonaggregated column unless it is grouped or otherwise allowed.

Common mistakes to avoid

- COUNT(*) counts rows; COUNT(column) ignores NULL values.
- WHERE filters rows; HAVING filters groups.

Quick viva and follow-up cues

- Core answer: Show grouping columns, aggregate input, and whether NULL rows contribute.
- Likely follow-up: COUNT(*) and COUNT(column) differ when the column contains NULL. Do not select a nonaggregated column unless it is grouped or otherwise allowed.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Show grouping columns, aggregate input, and whether NULL rows contribute.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 43 / 60 | Introduction on SQL CONCEPT AND DEFINITIONS

Joins

Simple definition

A join combines related rows from two or more tables using a matching condition.

Formal / technical definition

A join is a table operation that pairs rows according to a join condition. INNER JOIN keeps matches; LEFT/RIGHT/FULL OUTER JOIN additionally preserves unmatched rows from designated sides with NULL-extended columns; CROSS JOIN forms a Cartesian product.

Notation and how to read it

INNER: matched pairs; LEFT: all left + matches; FULL: all both sides; CROSS: every pair

How it works

- Write the participating tables and keys.
- State the ON condition and join type.
- Trace matched and unmatched rows and watch for one-to-many duplicates.

Quick map / trace / syntax

```
STUDENT rows + ENROLL rows
match s.SID=e.SID -> joined result row
```

QUICK RECALL

Name the join type, join predicate, and what happens to unmatched rows.

TOPIC 43 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Joins

Worked code example and output

```
SELECT s.Name, e.CID
FROM Student s
JOIN Enrollment e ON e.SID=s.SID;
```

Useful distinction / limitation

Missing or incorrect join predicates can create a huge Cartesian result or duplicate rows due to one-to-many matches.

Common mistakes to avoid

- A missing join condition can create a Cartesian product.
- Outer joins preserve unmatched rows from their specified side(s).

Quick viva and follow-up cues

- Core answer: Name the join type, join predicate, and what happens to unmatched rows.
- Likely follow-up: Missing or incorrect join predicates can create a huge Cartesian result or duplicate rows due to one-to-many matches.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Name the join type, join predicate, and what happens to unmatched rows.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 44 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

UNION

Simple definition

UNION combines result rows from two compatible queries and removes duplicates by default.

Formal / technical definition

UNION is a set operation requiring the same number of corresponding columns with compatible types. UNION ALL retains duplicates and may be faster because it does not deduplicate.

Notation and how to read it

Result = distinct rows in A or B; UNION ALL = bag concatenation

How it works

- Write each query result as a set of rows.
- Check compatible column count and types.
- Apply the set rule and account for dialect and duplicate semantics.

Quick map / trace / syntax

```
A rows + B rows -> combine -> duplicate elimination unless ALL
```

QUICK RECALL

Check column count/type alignment and decide whether duplicates should be removed.

TOPIC 44 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

UNION

Worked code example and output

```
SELECT SID FROM CurrentStudent
UNION
SELECT SID FROM Alumni;
```

Useful distinction / limitation

ORDER BY belongs at the end of the combined query in standard usage; each branch's output columns align by position.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Check column count/type alignment and decide whether duplicates should be removed.
- Likely follow-up: ORDER BY belongs at the end of the combined query in standard usage; each branch's output columns align by position.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Check column count/type alignment and decide whether duplicates should be removed.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 45 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

INTERSECTION

Simple definition

Intersection returns only rows that occur in both query results.

Formal / technical definition

The relational set intersection requires union-compatible operands. SQL INTERSECT is supported by many systems; duplicate behavior and INTERSECT ALL support vary by product.

Notation and how to read it

A INTERSECT B = rows in A and B

How it works

- Write each query result as a set of rows.
- Check compatible column count and types.
- Apply the set rule and account for dialect and duplicate semantics.

Quick map / trace / syntax

```
A: {1,2,3}; B: {2,3,4}
A INTERSECT B: {2,3}
```

QUICK RECALL

Explain compatibility requirements and identify the common rows.

TOPIC 45 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

INTERSECTION

Worked code example and output

```
SELECT SID FROM SportsClub
INTERSECT
SELECT SID FROM CSE_Students;
```

Useful distinction / limitation

If a DBMS lacks INTERSECT, equivalent EXISTS/IN logic can be used while handling duplicates deliberately.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Explain compatibility requirements and identify the common rows.
- Likely follow-up: If a DBMS lacks INTERSECT, equivalent EXISTS/IN logic can be used while handling duplicates deliberately.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Explain compatibility requirements and identify the common rows.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

MINUS

Simple definition

MINUS returns rows from the first result that do not appear in the second.

Formal / technical definition

MINUS is a set difference operator in some SQL dialects (notably Oracle); other systems commonly use EXCEPT. Operands must be union-compatible; duplicates are typically eliminated unless an ALL form is supported.

Notation and how to read it

A MINUS B = rows in A and not in B

How it works

- Write each query result as a set of rows.
- Check compatible column count and types.
- Apply the set rule and account for dialect and duplicate semantics.

Quick map / trace / syntax

```
A: {1,2,3}; B: {2,4}
A MINUS B: {1,3}
```

QUICK RECALL

The operation is directional: A MINUS B need not equal B MINUS A.

MINUS

Worked code example and output

```
SELECT SID FROM Applicants
MINUS
SELECT SID FROM Admitted;
-- Some DBMSs spell this EXCEPT
```

Useful distinction / limitation

MINUS is vendor-specific syntax; state the equivalent EXCEPT name when discussing portability.

Common mistakes to avoid

- MINUS is directional and not supported with that spelling by every DBMS.
- EXCEPT is the common alternative spelling; check dialect and duplicate rules.

Quick viva and follow-up cues

- Core answer: The operation is directional: A MINUS B need not equal B MINUS A.
- Likely follow-up: MINUS is vendor-specific syntax; state the equivalent EXCEPT name when discussing portability.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: The operation is directional: A MINUS B need not equal B MINUS A.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 47 / 60 | Introduction on SQL CONCEPT AND DEFINITIONS

Cursors

Simple definition

A cursor lets procedural database code process a query result row by row when a set-based statement is not suitable.

Formal / technical definition

A cursor is a control structure associated with a result set and a current position. Typical explicit-cursor stages are declare, open, fetch, process, and close; syntax is procedural-language and vendor dependent.

Notation and how to read it

DECLARE -> OPEN -> FETCH -> process -> FETCH ... -> CLOSE

How it works

- Declare and open a result-producing cursor.
- Fetch one row, process it, and test the end-of-data condition.
- Close the cursor and prefer a set-based alternative if possible.

Quick map / trace / syntax

```
result set: row1 -> row2 -> row3
^ current cursor position
```

QUICK RECALL

List cursor lifecycle stages and explain the current-row position.

TOPIC 47 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Cursors

Worked code example and output

```
Pseudo-code:
OPEN student_cursor;
FETCH student_cursor INTO sid, name;
WHILE a row is available: process it; FETCH again;
CLOSE student_cursor;
```

Useful distinction / limitation

Prefer set-based SQL for ordinary bulk work; row-by-row processing may be slower. Handle end-of-data and cleanup.

Common mistakes to avoid

- Always account for no-more-rows and close/cleanup.
- Prefer set-based SQL when it expresses the task clearly.

Quick viva and follow-up cues

- Core answer: List cursor lifecycle stages and explain the current-row position.
- Likely follow-up: Prefer set-based SQL for ordinary bulk work; row-by-row processing may be slower. Handle end-of-data and cleanup.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: List cursor lifecycle stages and explain the current-row position.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 48 / 60 | Introduction on SQL

CONCEPT AND DEFINITIONS

Triggers

Simple definition

A trigger is database code that runs automatically when a specified event happens on a table or view.

Formal / technical definition

A trigger associates an event (such as INSERT, UPDATE, or DELETE) and timing (BEFORE/AFTER/INSTEAD OF, depending on DBMS) with a procedural action. Row-level and statement-level behavior are product-specific.

Notation and how to read it

event + timing + scope + condition + action -> automatic execution

How it works

- Name the event, timing, table, and row/statement scope.
- Evaluate any trigger condition and run its action.
- Check side effects, recursion, and vendor-specific details.

Quick map / trace / syntax

```
INSERT/UPDATE/DELETE event -> trigger condition -> trigger body -> related action
```

QUICK RECALL

Name the triggering event, timing, affected table, and action; state that syntax differs by DBMS.

TOPIC 48 / 60 | Introduction on SQL

CODE EXAMPLE AND EXAM HELP

Triggers

Worked code example and output

```
Conceptual example: AFTER INSERT ON Enrollment, record an audit entry containing the
inserted SID and CID. Exact syntax varies by database product.
```

Useful distinction / limitation

Triggers can hide side effects and create recursion or ordering surprises. Keep the action focused and document it.

Common mistakes to avoid

- Triggers create automatic side effects that may be hidden from a caller.
- Declare event, timing, granularity, and product-specific syntax.

Quick viva and follow-up cues

- Core answer: Name the triggering event, timing, affected table, and action; state that syntax differs by DBMS.
- Likely follow-up: Triggers can hide side effects and create recursion or ordering surprises. Keep the action focused and document it.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Name the triggering event, timing, affected table, and action; state that syntax differs by DBMS.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 49 / 60 | Data Base Design & Normalization CONCEPT AND DEFINITIONS

Functional Dependencies

Simple definition

A functional dependency says that knowing some attribute values determines one or more other attribute values in every valid relation state.

Formal / technical definition

For relation schema R, X -> Y holds when any two legal tuples agreeing on X also agree on Y. Dependencies are semantic constraints, not facts inferred from one small sample table.

Notation and how to read it

X -> Y; trivial if Y subseteq X; nontrivial otherwise; closure X+ is attributes derivable from X and FDs

How it works

- Write the relation and the proposed X -> Y rule.
- Use domain semantics or a counterexample to test it.
- Compute closures/candidate keys when normalization requires them.

Quick map / trace / syntax

```
same X values in two rows -> same Y values
SID -> Name, Dept
```

QUICK RECALL

State the relation, determinant, dependent attributes, and assumption behind each FD.

TOPIC 49 / 60 | Data Base Design & Normalization

CODE EXAMPLE AND EXAM HELP

Functional Dependencies

Worked code example and output

```
Student(SID, Name, Dept): SID -> Name, Dept if each SID names one student. Dept -> Name usually does not hold because many students share a department.
```

Useful distinction / limitation

A sample table that happens to satisfy X -> Y does not prove the rule; use the real-world meaning.

Common mistakes to avoid

- A few sample rows do not prove an FD.
- Use business meaning and legal states to justify the dependency.

Quick viva and follow-up cues

- Core answer: State the relation, determinant, dependent attributes, and assumption behind each FD.
- Likely follow-up: A sample table that happens to satisfy X -> Y does not prove the rule; use the real-world meaning.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: State the relation, determinant, dependent attributes, and assumption behind each FD.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

Normal Forms

Simple definition

Normal forms are design checks that help reduce avoidable repetition and the insert, update, and delete problems it causes.

Formal / technical definition

A relation is in a normal form when its attributes and dependencies satisfy specified structural conditions. Normalization decomposes relations while aiming to preserve information and, when possible, dependencies.

Notation and how to read it

1NF -> atomic values (design convention); 2NF -> no partial dependency of non-prime attributes on candidate-key part; 3NF -> dependency condition; BCNF -> every nontrivial determinant is a superkey

How it works

- List candidate keys, prime attributes, and relevant dependencies.
- Apply the exact normal-form definition to every nontrivial dependency.
- Decompose, then test losslessness and dependency preservation.

Quick map / trace / syntax

```
repeating facts -> anomalies -> analyze dependencies -> decompose -> test
losslessness/dependency preservation
```

QUICK RECALL

Define the target normal form precisely and identify which dependency causes a violation.

TOPIC 50 / 60 | Data Base Design & Normalization CODE EXAMPLE AND EXAM HELP

Normal Forms

Worked code example and output

```
Repeating a department phone on every employee row can cause inconsistent updates.
Separating Department(DeptID, Phone) removes that repeated fact.
```

Useful distinction / limitation

Higher normal form is not automatically a better practical schema if performance or query simplicity is ignored; correctness tests still matter.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Define the target normal form precisely and identify which dependency causes a violation.
- Likely follow-up: Higher normal form is not automatically a better practical schema if performance or query simplicity is ignored; correctness tests still matter.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Define the target normal form precisely and identify which dependency causes a violation.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 51 / 60 | Data Base Design & Normalization CONCEPT AND DEFINITIONS

First Normal Form (1NF)

Simple definition

A 1NF table stores one value in each cell for the chosen attribute meaning; lists packed into one cell make searching and validation difficult.

Formal / technical definition

Under the common relational design convention, attributes hold atomic values from their domains and repeating groups are removed. Atomicity is relative to the intended operations and domain semantics.

Notation and how to read it

one row-column intersection -> one domain value; no repeating groups

How it works

- Identify repeating groups or non-atomic values for the intended meaning.
- Create a row/attribute representation with one domain value per cell.
- Preserve identity and relationship information with keys.

Quick map / trace / syntax

```
Before: SID | Phones
17 | 111, 222
After: SID | Phone
17 | 111; 17 | 222
```

QUICK RECALL

Show the repeating group and a relation design with one phone value per row.

TOPIC 51 / 60 | Data Base Design & Normalization

CODE EXAMPLE AND EXAM HELP

First Normal Form (1NF)

Worked code example and output

```
Not 1NF design: Student(SID, Phones='111,222'). 1NF decomposition: Student(SID, Name);
StudentPhone(SID, Phone).
```

Useful distinction / limitation

Atomic does not mean physically indivisible bytes; it means one value for the modeled attribute and use.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Show the repeating group and a relation design with one phone value per row.
- Likely follow-up: Atomic does not mean physically indivisible bytes; it means one value for the modeled attribute and use.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Show the repeating group and a relation design with one phone value per row.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 52 / 60 | Data Base Design & Normalization CONCEPT AND DEFINITIONS

Second Normal Form (2NF)

Simple definition

2NF removes a dependency where a non-key fact depends on only part of a multi-column candidate key.

Formal / technical definition

A relation is in 2NF if it is in 1NF and each non-prime attribute is fully functionally dependent on every candidate key; no non-prime attribute may depend on a proper subset of a candidate key.

Notation and how to read it

partial dependency: proper subset of candidate key -> non-prime attribute

How it works

- List candidate keys, prime attributes, and relevant dependencies.
- Apply the exact normal-form definition to every nontrivial dependency.
- Decompose, then test losslessness and dependency preservation.

Quick map / trace / syntax

```
(SID,CID) -> Grade; SID -> StudentName
StudentName repeats for each course -> remove partial dependency
```

QUICK RECALL

Find every candidate key, identify non-prime attributes, and test proper subsets of each key.

TOPIC 52 / 60 | Data Base Design & Normalization

CODE EXAMPLE AND EXAM HELP

Second Normal Form (2NF)

Worked code example and output

```
Enrollment(SID,CID,StudentName,Grade), key (SID,CID), and SID -> StudentName. StudentName
depends only on part SID, so split Student(SID,StudentName) and Enrollment(SID,CID,Grade).
```

Useful distinction / limitation

2NF matters when a candidate key is composite. A single-attribute candidate key cannot have a proper nonempty subset causing this partial dependency.

Common mistakes to avoid

- 2NF concerns partial dependency on part of a composite candidate key.
- Test all candidate keys and only non-prime attributes for the formal condition.

Quick viva and follow-up cues

- Core answer: Find every candidate key, identify non-prime attributes, and test proper subsets of each key.
- Likely follow-up: 2NF matters when a candidate key is composite. A single-attribute candidate key cannot have a proper nonempty subset causing this partial dependency.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Find every candidate key, identify non-prime attributes, and test proper subsets of each key.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 53 / 60 | Data Base Design & Normalization CONCEPT AND DEFINITIONS

Third Normal Form (3NF)

Simple definition

3NF removes certain transitive dependencies so non-key facts do not depend on another non-key fact.

Formal / technical definition

A relation is in 3NF if for every nontrivial FD X -> A, either X is a superkey or A is prime (belongs to some candidate key). Equivalently, after 2NF, there is no prohibited transitive dependency of a non-prime attribute on a key.

Notation and how to read it

For every nontrivial X -> A: X superkey OR A prime

How it works

- List candidate keys, prime attributes, and relevant dependencies.
- Apply the exact normal-form definition to every nontrivial dependency.
- Decompose, then test losslessness and dependency preservation.

Quick map / trace / syntax

```
EID -> DeptID -> DeptName
Move DeptName to Department so the department fact is stored once
```

QUICK RECALL

Apply the formal test to every nontrivial FD and verify lossless reconstruction.

TOPIC 53 / 60 | Data Base Design & Normalization

CODE EXAMPLE AND EXAM HELP

Third Normal Form (3NF)

Worked code example and output

```
Employee(EID, DeptID, DeptName): EID -> DeptID and DeptID -> DeptName, so EID -> DeptName
transitively. Split Department(DeptID,DeptName) from Employee(EID,DeptID).
```

Useful distinction / limitation

The formal prime-attribute condition is safer than the simplified phrase 'no transitive dependency'; some transitive dependencies are allowed.

Common mistakes to avoid

- Use the formal determinant-superkey OR dependent-prime test.
- Do not treat every transitive dependency as a 3NF violation without checking prime status.

Quick viva and follow-up cues

- Core answer: Apply the formal test to every nontrivial FD and verify lossless reconstruction.
- Likely follow-up: The formal prime-attribute condition is safer than the simplified phrase 'no transitive dependency'; some transitive dependencies are allowed.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Apply the formal test to every nontrivial FD and verify lossless reconstruction.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 54 / 60 | Data Base Design & Normalization

CONCEPT AND DEFINITIONS

Boyce-Codd Normal Form (BCNF)

Simple definition

BCNF is stricter than 3NF: every determinant that determines new information must be a superkey.

Formal / technical definition

A relation R is in BCNF if for every nontrivial functional dependency X -> Y that holds in R, X is a superkey of R.

Notation and how to read it

nontrivial X -> Y implies X is a superkey

How it works

- List candidate keys, prime attributes, and relevant dependencies.
- Apply the exact normal-form definition to every nontrivial dependency.
- Decompose, then test losslessness and dependency preservation.

Quick map / trace / syntax

```
Instructor -> Course
Instructor does not identify every R tuple -> BCNF violation
```

QUICK RECALL

List the FDs, find each determinant's closure, and compare it with all relation attributes.

TOPIC 54 / 60 | Data Base Design & Normalization

CODE EXAMPLE AND EXAM HELP

Boyce-Codd Normal Form (BCNF)

Worked code example and output

```
R(Student,Course,Instructor), with (Student,Course)->Instructor and Instructor->Course.
Instructor is not a superkey, so the second FD violates BCNF. Decompose
InstructorCourse(Instructor,Course) and StudentInstructor(Student,Instructor), checking
losslessness under the stated FDs.
```

Useful distinction / limitation

A 3NF relation can fail BCNF when a determinant is not a superkey but the dependent attribute is prime.

Common mistakes to avoid

- For each nontrivial FD, test whether the determinant is a superkey.
- 3NF does not always imply BCNF.

Quick viva and follow-up cues

- Core answer: List the FDs, find each determinant's closure, and compare it with all relation attributes.
- Likely follow-up: A 3NF relation can fail BCNF when a determinant is not a superkey but the dependent attribute is prime.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: List the FDs, find each determinant's closure, and compare it with all relation attributes.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 55 / 60 | Data Base Design & Normalization CONCEPT AND DEFINITIONS

Inclusion Dependence

Simple definition

An inclusion dependency says that values appearing in selected columns of one relation must also appear in selected columns of another.

Formal / technical definition

For relation schemas R and S, R[X] subseteq S[Y] requires every projection of an R tuple on X to occur as a projection of some S tuple on Y. Foreign-key constraints are a common special case when the referenced attributes are a key.

Notation and how to read it

R[X] subseteq S[Y]; FK child columns -> referenced parent key columns

How it works

- List candidate keys, prime attributes, and relevant dependencies.
- Apply the exact normal-form definition to every nontrivial dependency.
- Decompose, then test losslessness and dependency preservation.

Quick map / trace / syntax

```
values(Enrollment.SID) subset of values(Student.SID)
```

QUICK RECALL

Name source and target relations and align each included attribute position with its counterpart.

TOPIC 55 / 60 | Data Base Design & Normalization

CODE EXAMPLE AND EXAM HELP

Inclusion Dependence

Worked code example and output

```
Enrollment[SID] subseteq Student[SID] says every enrolled SID must exist as a student,
when the FK is non-NULL.
```

Useful distinction / limitation

Inclusion dependencies can express matching without asserting that the child columns uniquely identify rows.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Name source and target relations and align each included attribute position with its counterpart.
- Likely follow-up: Inclusion dependencies can express matching without asserting that the child columns uniquely identify rows.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Name source and target relations and align each included attribute position with its counterpart.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 56 / 60 | Data Base Design & Normalization CONCEPT AND DEFINITIONS

Lossless Join Decompositions

Simple definition

A decomposition is lossless if joining the smaller tables recreates exactly the original valid rows, without inventing extra combinations.

Formal / technical definition

A decomposition of R into R1 and R2 is lossless with respect to an instance/constraints when their natural join equals R (for every legal state under the dependencies). For binary decomposition and FDs, a common test is that R1 intersect R2 functionally determines R1 or R2.

Notation and how to read it

R1 join R2 = R (no spurious tuples); binary FD test: (R1 intersect R2)->R1 OR ->R2

How it works

- List the decomposed schemas and their shared attributes.
- Apply the binary FD test or use a chase for a larger decomposition.
- State why no spurious tuples appear and distinguish dependency preservation.

Quick map / trace / syntax

```
lossless: original tuples reconstruct
lossy: join produces an extra/spurious tuple
```

QUICK RECALL

Compute the intersection, test the FD condition, and explain what a spurious tuple would mean.

TOPIC 56 / 60 | Data Base Design & Normalization

CODE EXAMPLE AND EXAM HELP

Lossless Join Decompositions

Worked code example and output

```
R(A,B,C), FD A->B. Decompose AB and AC; common attribute A determines AB, so the binary FD test gives a lossless join.
```

Useful distinction / limitation

Losslessness and dependency preservation are separate properties; a decomposition may be lossless but require joins to check some dependencies.

Common mistakes to avoid

- Losslessness does not imply dependency preservation.
- A lossy join can create tuples that were not in the original relation.

Quick viva and follow-up cues

- Core answer: Compute the intersection, test the FD condition, and explain what a spurious tuple would mean.
- Likely follow-up: Losslessness and dependency preservation are separate properties; a decomposition may be lossless but require joins to check some dependencies.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Compute the intersection, test the FD condition, and explain what a spurious tuple would mean.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 57 / 60 | Data Base Design & Normalization CONCEPT AND DEFINITIONS

Normalization Using Functional Dependencies

Simple definition

FD-based normalization uses determinants and candidate keys to split a wide table into smaller tables with fewer update anomalies.

Formal / technical definition

A decomposition algorithm derives candidate keys/closures and applies synthesis (often producing a dependency-preserving 3NF schema) or decomposition (often seeking BCNF and losslessness). Results depend on a correct minimal cover and the specified FD set.

Notation and how to read it

Typical steps: find attribute closures -> candidate keys -> minimal cover -> synthesize/decompose -> test losslessness and dependency preservation

How it works

- List candidate keys, prime attributes, and relevant dependencies.
- Apply the exact normal-form definition to every nontrivial dependency.
- Decompose, then test losslessness and dependency preservation.

Quick map / trace / syntax

```
FD set -> keys/minimal cover -> relation schemas -> verify join and constraints
```

QUICK RECALL

Show each closure/key calculation and state whether the method aims at 3NF or BCNF.

TOPIC 57 / 60 | Data Base Design & Normalization

CODE EXAMPLE AND EXAM HELP

Normalization Using Functional Dependencies

Worked code example and output

```
R(A,B,C), F={A->B, B->C}. 3NF synthesis can create AB and BC; shared B joins them
losslessly because B->BC. The FDs remain checkable within relations.
```

Useful distinction / limitation

Do not decompose from repeated-looking values alone; use the declared dependencies and verify the output relations.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Show each closure/key calculation and state whether the method aims at 3NF or BCNF.
- Likely follow-up: Do not decompose from repeated-looking values alone; use the declared dependencies and verify the output relations.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Show each closure/key calculation and state whether the method aims at 3NF or BCNF.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 58 / 60 | Data Base Design & Normalization CONCEPT AND DEFINITIONS

Normalization Using Multivalued Dependencies (MVD)

Simple definition

An MVD says that two sets of values vary independently for each determinant; storing both independent lists together creates combinations and repetition.

Formal / technical definition

A multivalued dependency X ->> Y holds when, for a fixed X, Y-values can be recombined independently of the remaining attributes Z=R-(X union Y). Fourth normal form requires each nontrivial MVD X ->> Y to have X as a superkey.

Notation and how to read it

nontrivial X ->> Y; 4NF: X superkey for each nontrivial MVD

How it works

- Identify independently varying attribute sets for each determinant.
- Show the repeated combinations in the original relation.
- Decompose and check that the join recreates exactly the legal rows.

Quick map / trace / syntax

```
Student skills {SQL, Python} x languages {English, Hindi}
combined rows repeat all independent pairings
```

QUICK RECALL

State the independent sets, show the cross-product redundancy, and test whether the determinant is a superkey.

TOPIC 58 / 60 | Data Base Design & Normalization CODE EXAMPLE AND EXAM HELP

Normalization Using Multivalued Dependencies (MVD)

Worked code example and output

```
StudentSkillLanguage(Student,Skill,Language): if a student's skills and languages are independent, Student ->> Skill and Student ->> Language. Decompose to StudentSkill(Student,Skill) and StudentLanguage(Student,Language).
```

Useful distinction / limitation

Do not infer an MVD from a few crossed rows; it is an independent-combination constraint about every legal state.

Common mistakes to avoid

- Do not confuse a functional dependency with independent multivalued facts.
- A nontrivial MVD violates 4NF when its determinant is not a superkey.

Quick viva and follow-up cues

- Core answer: State the independent sets, show the cross-product redundancy, and test whether the determinant is a superkey.
- Likely follow-up: Do not infer an MVD from a few crossed rows; it is an independent-combination constraint about every legal state.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: State the independent sets, show the cross-product redundancy, and test whether the determinant is a superkey.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 59 / 60 | Data Base Design & Normalization CONCEPT AND DEFINITIONS

Normalization Using Join Dependencies (JD)

Simple definition

A join dependency says a table can be reconstructed exactly by joining several projections; it can reveal redundancy beyond FDs and MVDs.

Formal / technical definition

A JD *(R1,R2,...,Rn) holds on R when every legal instance r equals the join of its projections on Ri. A nontrivial JD not implied by candidate keys motivates fifth normal form (5NF / project-join normal form).

Notation and how to read it

JD *(R1,...,Rn): r = pi_R1(r) join ... join pi_Rn(r); 5NF handles key-related nontrivial JDs

How it works

- List candidate keys, prime attributes, and relevant dependencies.
- Apply the exact normal-form definition to every nontrivial dependency.
- Decompose, then test losslessness and dependency preservation.

Quick map / trace / syntax

```
original relation -> project to several schemas -> natural join -> original relation exactly
```

QUICK RECALL

State the projection schemas and prove that their join returns exactly the original relation.

TOPIC 59 / 60 | Data Base Design & Normalization CODE EXAMPLE AND EXAM HELP

Normalization Using Join Dependencies (JD)

Worked code example and output

```
Supplier-Part-Project(S,P,J) may, under a specific business rule, equal SupplierPart(S,P)
join SupplierProject(S,J) join PartProject(P,J). This rule must be valid for the domain,
not assumed from a sample.
```

Useful distinction / limitation

A JD is stronger/more general than an MVD; arbitrary decomposition into binary relationships can introduce spurious combinations.

Common mistakes to avoid

- A JD is about equality with the join of projections for every legal state.
- Do not assert a JD from one sample instance.

Quick viva and follow-up cues

- Core answer: State the projection schemas and prove that their join returns exactly the original relation.
- Likely follow-up: A JD is stronger/more general than an MVD; arbitrary decomposition into binary relationships can introduce spurious combinations.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: State the projection schemas and prove that their join returns exactly the original relation.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

TOPIC 60 / 60 | Data Base Design & Normalization CONCEPT AND DEFINITIONS

Alternative Approaches to Database Design

Simple definition

A database can be designed from requirements downward, from existing data upward, or by combining both approaches and then checking the result.

Formal / technical definition

Top-down design identifies entities, relationships, and constraints from requirements before mapping to relations. Bottom-up design analyzes attributes and dependencies to synthesize relations. Hybrid workflows use conceptual modeling, normalization, workload review, and iterative validation.

Notation and how to read it

Requirements -> conceptual model -> logical relational design -> physical tuning -> validation

How it works

- List candidate keys, prime attributes, and relevant dependencies.
- Apply the exact normal-form definition to every nontrivial dependency.
- Decompose, then test losslessness and dependency preservation.

Quick map / trace / syntax

```
Top-down gives a conceptual overview; bottom-up exposes dependencies in detailed attributes;
both require constraint and query review.
```

QUICK RECALL

Compare top-down and bottom-up, then outline a hybrid process and its validation checks.

TOPIC 60 / 60 | Data Base Design & Normalization CODE EXAMPLE AND EXAM HELP

Alternative Approaches to Database Design

Worked code example and output

```
Top-down: interview users -> ER/EER model -> relational schema -> normalize. Bottom-up:
collect fields -> infer FDs -> form relation schemas -> validate against user workflows.
```

Useful distinction / limitation

Good design also checks lossless joins, dependency preservation, keys, nullability, required queries, and realistic data examples.

Common mistakes to avoid

- State the topic's rule before applying it.
- Trace the data or control flow carefully; check the final value and type.

Quick viva and follow-up cues

- Core answer: Compare top-down and bottom-up, then outline a hybrid process and its validation checks.
- Likely follow-up: Good design also checks lossless joins, dependency preservation, keys, nullability, required queries, and realistic data examples.

Exam answer skeleton

- Define the topic and name the exact Python syntax.
- Trace the example and state its output/result.
- Close with this rule or limitation: Compare top-down and bottom-up, then outline a hybrid process and its validation checks.

QUICK RECALL

Try the example once without looking, then explain why the result is correct.

Rapid revision: MODULE-I definitions (1/2)

Introduction

- An Overview of Database Management System: A database stores related information in an organized way.
- Database System vs File System: With separate files, every program manages its own data.
- Database System Concept and Architecture: Database architecture explains how users see data and how the system stores and processes it behind the scenes.
- Data Model, Schema, and Instances: A data model is a set of ideas for describing data.
- Data Independence: Data independence means changing one level of database design without forcing every higher-level view or program to change.
- Database Language and Interfaces: Users work with a DBMS through languages and interfaces: SQL, forms, graphical tools, APIs, or administrative consoles.
- Data Definition Language (DDL): DDL statements create or change database structures such as tables, constraints, and views.
- Data Manipulation Language (DML): DML works with the rows inside tables: it retrieves, adds, changes, or removes data.
- Overall Database Structure: A DBMS has parts that understand requests, find data, manage storage, enforce rules, and recover after failures.

Data Modeling

- ER Data Model: The Entity-Relationship model is a picture-based way to plan a database: it shows things we store, their properties, and how they connect.
- Notation for ER Diagram: ER notation uses shapes and labels so a diagram tells us which parts are entities, attributes, and relationships.
- Mapping Constraints: Mapping constraints say how many entities can be connected and whether taking part is optional or required.
- Super Key, Candidate Key, and Primary Key: A key identifies a row.
- Generalization: Generalization combines several similar lower-level entity types into one common parent type.
- Aggregation: Aggregation lets an ER model treat a relationship and its participating entities as one higher-level object so another relationship can refer to it.
- Reduction of an ER Diagram to Tables: ER-to-table mapping turns the conceptual diagram into relational tables while preserving entities, keys, and relationships.
- Extended ER (EER) Model: EER adds richer ideas to ER diagrams, especially subclasses, inheritance, and constraints on specialization.
- Relationship of Higher Degree: A higher-degree relationship connects three or more entity types at once; the relationship meaning depends on the whole combination.

Rapid revision: MODULE-I definitions (2/2)

Relational Data Model

- Relational Data Model Concepts: The relational model stores data in relations, usually shown as tables with rows and columns.
- Integrity Constraints: Integrity constraints are rules that keep stored data valid and consistent.
- Entity Integrity: Entity integrity says a primary-key value must exist for every row, so each row can be identified.
- Referential Integrity: Referential integrity prevents a reference from pointing to a row that does not exist.
- Key Constraints: Key constraints require chosen attribute values, or combinations of them, to identify rows uniquely.

Relational rules and query formalisms

- Domain Constraints: A domain constraint limits a column to values of the right kind and range.
- Relational Algebra: Relational algebra is a step-by-step language for querying relations.
- Relational Calculus: Relational calculus describes what result is wanted by stating logical conditions, rather than listing a sequence of table operations.
- Tuple Relational Calculus: Tuple relational calculus uses tuple variables: each variable stands for a whole row.
- Domain Relational Calculus: Domain relational calculus uses variables for individual column values rather than whole rows.

Rapid revision: MODULE-II definitions (1/2)

SQL foundations

- Characteristics of SQL: SQL is the common language used to define relational database objects and work with their data.
- Advantage of SQL: SQL gives users a standard, concise way to ask questions about relational data without writing low-level file-processing code.
- SQL Data Types and Literals: A column's type describes what values it stores.
- Types of SQL Commands: SQL commands are often grouped by whether they define structures, manipulate data, control permissions, or manage transactions.
- SQL Operators: Operators compare or combine values so a query can calculate, filter, or match rows.
- Tables: A table is a named structure that stores rows with the same set of columns.
- Views: A view is a named query that presents selected or combined data like a virtual table.
- Indexes: An index is an extra access structure that can help the DBMS find rows faster, usually at the cost of storage and slower writes.
- INSERT Operation: INSERT adds one or more rows to a table.
- UPDATE Operation: UPDATE changes values in existing rows that satisfy a condition.

SQL operations and query tools

- DELETE Operation: DELETE removes selected rows.
- Queries: A query asks the database to return rows or calculated values that match specified conditions.
- Subqueries: A subquery is a query placed inside another SQL statement; its result helps the outer query decide what to return or change.
- Aggregate Functions: Aggregate functions summarize many rows into one value, such as a count, average, smallest value, or largest value.
- Joins: A join combines related rows from two or more tables using a matching condition.
- UNION: UNION combines result rows from two compatible queries and removes duplicates by default.
- INTERSECTION: Intersection returns only rows that occur in both query results.
- MINUS: MINUS returns rows from the first result that do not appear in the second.
- Cursors: A cursor lets procedural database code process a query result row by row when a set-based statement is not suitable.
- Triggers: A trigger is database code that runs automatically when a specified event happens on a table or view.

Rapid revision: MODULE-II definitions (2/2)

Dependencies and normal forms

- Functional Dependencies: A functional dependency says that knowing some attribute values determines one or more other attribute values in every valid relation state.
- Normal Forms: Normal forms are design checks that help reduce avoidable repetition and the insert, update, and delete problems it causes.
- First Normal Form (1NF): A 1NF table stores one value in each cell for the chosen attribute meaning; lists packed into one cell make searching and validation difficult.
- Second Normal Form (2NF): 2NF removes a dependency where a non-key fact depends on only part of a multi-column candidate key.
- Third Normal Form (3NF): 3NF removes certain transitive dependencies so non-key facts do not depend on another non-key fact.
- Boyce-Codd Normal Form (BCNF): BCNF is stricter than 3NF: every determinant that determines new information must be a superkey.

Decomposition and design

- Inclusion Dependence: An inclusion dependency says that values appearing in selected columns of one relation must also appear in selected columns of another.
- Lossless Join Decompositions: A decomposition is lossless if joining the smaller tables recreates exactly the original valid rows, without inventing extra combinations.
- Normalization Using Functional Dependencies: FD-based normalization uses determinants and candidate keys to split a wide table into smaller tables with fewer update anomalies.
- Normalization Using Multivalued Dependencies (MVD): An MVD says that two sets of values vary independently for each determinant; storing both independent lists together creates combinations and repetition.
- Normalization Using Join Dependencies (JD): A join dependency says a table can be reconstructed exactly by joining several projections; it can reveal redundancy beyond FDs and MVDs.
- Alternative Approaches to Database Design: A database can be designed from requirements downward, from existing data upward, or by combining both approaches and then checking the result.

ER, relational algebra, and calculus reference

ER-to-relation mapping

- Strong entity -> relation; simple attributes -> columns; key -> primary key.
- Composite attribute -> store components; derived attribute -> usually compute when needed.
- Multivalued attribute -> separate relation with owner key + value.
- 1:N -> FK on N-side; M:N -> relation with both FKs and relationship attributes.
- Weak entity -> relation includes owner key + partial key; owner FK is commonly part of its PK.

Cardinality and participation

- 1:1: FK on a suitable side with UNIQUE; total participation may require NOT NULL.
- 1:N: FK on the many side; M:N: associative relation.
- Cardinality is the maximum; min-max/participation records lower bounds.
- Higher-degree relation: create a relation containing participating entity keys and relationship attributes.

Relational algebra

- sigma condition(R): select rows. pi A,B(R): select columns.
- rho S(R): rename. R x S: Cartesian product.
- R union S / R-S / R intersect S require union-compatible schemas.
- R bowtie condition S: join; natural join matches same-named attributes (use carefully).

Relational calculus

- Tuple calculus: {t | P(t)}; t denotes a tuple.
- Domain calculus: {<x1,...,xn> | P}; x variables denote attribute values.
- exists expresses at least one witness; for all expresses every value.
- Safe/range-restricted queries avoid an unbounded infinite result.

Database constraints and design reference

Key and integrity checklist

- Candidate key = minimal superkey; primary key = selected candidate key.
- Entity integrity: primary-key components are NOT NULL and key combination is unique.
- Referential integrity: non-NULL FK value matches a referenced key (subject to declared rules).
- Domain constraint: each value has an allowed type/range/set.
- Key constraint: no two legal tuples share the full key value.

Schema and architecture

- External schema: user/application view.
- Conceptual schema: logical entities, relationships, constraints.
- Internal schema: files, pages, indexes, access paths.
- Physical data independence hides internal changes; logical independence hides selected conceptual changes.

Relation vocabulary

- Attribute = named column; tuple = row; domain = allowed values.
- Degree = attribute count; cardinality = tuple count.
- A relation schema is its heading; a relation instance is its current extension.
- Classical relations are sets; SQL results may contain duplicates unless DISTINCT/constraints prevent them.

Calculus vs algebra

- Relational algebra: procedural sequence of relational operations.
- Relational calculus: declarative predicate describing desired tuples.
- Tuple calculus binds row variables; domain calculus binds column-value variables.
- For safe relational queries, both formalisms express relationally complete queries.

SQL command and query reference

DDL / DML / DCL / TCL

- DDL: CREATE, ALTER, DROP schema objects.
- DML: INSERT, UPDATE, DELETE rows; SELECT often grouped here.
- DQL: SELECT as a separate classroom category in many texts.
- DCL: GRANT / REVOKE privileges. TCL: COMMIT / ROLLBACK / SAVEPOINT.

SELECT pattern

- SELECT columns or expressions FROM source tables.
- JOIN ... ON combines related rows; WHERE filters rows.
- GROUP BY forms groups; HAVING filters groups.
- ORDER BY sorts final output; ordering is not guaranteed without it.

Set operations

- UNION combines and deduplicates; UNION ALL keeps duplicates.
- INTERSECT returns rows common to both results.
- MINUS (Oracle) / EXCEPT (common alternative) returns first-result rows missing in second.
- Operands need compatible column counts/types; output columns align by position.

DML safety

- INSERT: list target columns and matching values.
- UPDATE: SET expressions; WHERE limits target rows.
- DELETE: WHERE selects rows; without it all rows are deleted.
- Check constraints, foreign keys, and affected-row count.

Functional dependency and normal-form reference

FD and key calculations

- X -> Y means same X forces same Y in every legal state.
- X+ starts with X; repeatedly add attributes Y when an FD determinant is contained in the closure.
- X is a superkey if X+ contains all relation attributes; candidate key if minimal.
- Minimal cover: singleton RHS, no extraneous LHS attributes, no redundant FDs.

Normal-form tests

- 1NF: atomic values/no repeating groups (design convention).
- 2NF: 1NF; no non-prime attribute depends on a proper subset of any candidate key.
- 3NF: each nontrivial X -> A has X superkey OR A prime.
- BCNF: every nontrivial X -> Y has X superkey.

Higher dependencies

- MVD X ->> Y is nontrivial when Y not subseteq X and X union Y != R.
- 4NF: for every nontrivial MVD X ->> Y, X is a superkey.
- JD *(R1,...,Rn): relation equals join of its projections.
- 5NF handles nontrivial join dependencies not implied by keys.

Decomposition checks

- Lossless: join returns exactly original legal relation; no spurious tuples.
- For binary FD decomposition, intersection determines at least one side.
- Dependency preservation: constraints can be checked on component relations without joining.
- Losslessness and preservation are different properties.

Frequently asked exam and viva themes

MODULE-I - foundations and architecture

- Explain DBMS purpose, file-system limits, architecture, schema/instance, and independence.
- Draw ER/EER diagrams with keys, participation, cardinality, and degree.
- Map ER designs to tables and state the integrity constraints.

MODULE-I - relational theory

- Compute relational algebra expressions step by step.
- Compare algebra with calculus; write tuple and domain calculus.
- Find superkeys, candidate keys, and constraint violations.

MODULE-II - SQL

- Classify commands; write correct DDL/DML and safe update/delete statements.
- Trace joins, subqueries, aggregates, and set operations.
- Explain cursor lifecycle and trigger event/timing/action.

MODULE-II - database design

- Compute attribute closures and candidate keys from FDs.
- Apply 1NF, 2NF, 3NF, and BCNF definitions precisely.
- Check lossless join, dependency preservation, MVDs, and JDs.

How to earn method marks

- Write the rule before applying it.
- Show keys, constraints, intermediate relations, or query result rows.
- Name assumptions and DBMS dialect when syntax differs.
- For decomposition, explicitly test reconstruction and dependencies.

Viva response pattern

- Give one clear definition.
- State notation or syntax.
- Use a tiny example and trace it.
- Mention one limitation or common trap.

Module 1 exam practice and answer cues

Frequently asked long-answer questions

- Explain the three-schema architecture and distinguish schema from instance.
- Compare a database system with a file-processing system using clear criteria.
- Draw an ER model for a college registration system with keys and cardinalities.
- Reduce an ER diagram containing 1:N and M:N relationships to tables.
- Explain entity, referential, key, and domain integrity with examples.
- Compare relational algebra with tuple and domain relational calculus.

Short-answer / viva

- What is a DBMS?
- What is the difference between physical and logical data independence?
- Can two different schemas have the same instance?
- What is a weak entity?
- How does an M:N relationship map to tables?
- What is the difference between tuple and domain calculus?

Easy

- Identify relation, tuple, attribute, and domain in Student(SID,Name,Dept).
- Give one candidate key and one nonminimal superkey for a sample relation.
- Classify a foreign key and a CHECK constraint.
- Write one relational algebra selection and projection.

Moderate

- Draw Department 1:N Employee with optional/mandatory constraints.
- Convert an M:N Student-Course relationship with Grade into relations.
- Use an FD to show whether an attribute set is a superkey.
- Write a tuple-calculus expression for students in CSE.

Hard

- Design an EER model with disjoint/overlapping and total/partial constraints.
- Show why a ternary relationship cannot always be replaced by three binary relationships.
- Construct a safe tuple/domain calculus query and explain its range restriction.

Answer cues / final checks

- Degree counts columns; cardinality counts tuples.
- A primary key is selected from candidate keys; candidate keys are minimal superkeys.
- Cardinality maximum and participation optionality are separate constraints.
- A foreign key references a parent key; nullable FK values may represent an absent relationship.
- Algebra is procedural; calculus is declarative.

Module 2 exam practice and answer cues

Frequently asked long-answer questions

- Explain SQL command classes with examples and note common SELECT classification conventions.
- Trace a SELECT query with JOIN, WHERE, GROUP BY, HAVING, and ORDER BY.
- Compare INNER and OUTER joins; show what happens to unmatched rows.
- Explain functional dependencies, candidate keys, and attribute closure.
- Normalize a relation to 3NF or BCNF and test the decomposition.
- Explain lossless joins, dependency preservation, MVDs, and join dependencies.

Short-answer / viva

- What does NULL mean in SQL?
- Why is WHERE different from HAVING?
- What is a correlated subquery?
- What is a trigger?
- What is a cursor lifecycle?
- State the 3NF and BCNF tests.

Easy

- Classify CREATE, INSERT, GRANT, and COMMIT.
- Write an INSERT with an explicit column list.
- Count rows and non-NULL values with aggregate functions.
- Give an example of an update predicate.

Moderate

- Write a query for department averages with a HAVING condition.
- Use EXISTS to find students with at least one enrollment.
- Compute X+ from a supplied FD set and identify candidate keys.
- Find and remove a partial dependency in an enrollment table.

Hard

- Decompose a relation to BCNF and discuss whether dependencies remain local.
- Use MVD semantics to remove independent multi-valued facts.
- Analyze a three-relation JD and explain how spurious tuples are avoided.

Answer cues / final checks

- COUNT(*) counts rows; COUNT(col) ignores NULL.
- UPDATE/DELETE without WHERE affects all rows.
- 3NF permits X not a superkey when dependent attribute is prime; BCNF does not.
- Lossless join and dependency preservation are separate.
- Use dialect-aware syntax for MINUS/EXCEPT, cursors, and triggers.

Additional / Outside Syllabus: practical SQL habits

Habits for safer SQL

- Use explicit column lists in INSERT statements and parameterized queries in applications.
- Before UPDATE or DELETE, run a SELECT with the same predicate and inspect the intended rows.
- Use constraints for durable integrity rules instead of relying only on application screens.
- Use transactions for related changes when the application/database supports them; commit only after checking success.
- These are practical habits added for learning and are not extra syllabus headings.

Portable SQL checklist

- Check whether data types, date literals, identity columns, and string functions differ by DBMS.
- MINUS is not universal; EXCEPT is a common alternative. Trigger and cursor syntax is often vendor-specific.
- Use ORDER BY when displayed ordering matters; row storage order is not a guarantee.

QUICK RECALL

Understand the data change before running the statement.

Additional / Outside Syllabus: design review checklist

Review a relational design

- List entity facts, relationship rules, optionality, and business keys from the requirements.
- Write the candidate keys and functional dependencies that the organization promises to uphold.
- Check domain, NOT NULL, uniqueness, and foreign-key constraints.
- After decomposition, test losslessness and dependency preservation separately.
- Check that frequently required queries can be expressed clearly; physical tuning comes after logical correctness.

A small FD trace

- R(A,B,C), F={A->B, B->C}. Start A+={A}; apply A->B to get {A,B}; apply B->C to get {A,B,C}.
- Therefore A is a superkey. If no proper subset of A exists, it is a candidate key.
- This is an optional worked recap; all extra study guidance is labeled outside syllabus.

QUICK RECALL

Write each closure step explicitly; do not jump straight to the conclusion.

Created by Anurag Yadav

Email: annuanuragyadav908@gmail.com